# SMART GYM MANAGEMENT SYSTEM

**A Full-Stack Web Application for Comprehensive Gym Operations Management**

**(React + Spring Boot + Oracle)**


Submitted by:

**SUTHAR ARYAN SUJALKUMAR**

Enrollment No: 23C25512


Under the guidance of

**Dr. Ashutosh Abhangi**

*Project report submitted*

*In Partial Fulfillment of the Requirements for the Degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**INFORMATION TECHNOLOGY**

May 2026

BARODA UNIVERSITY
Vadodara | Gujarat | India
"Think Big... Think Beyond"


**School of Computer Science Engineering & Technology**

**ITM SLS Baroda University**

**Paldi Village, Vadodara – 390510**

# Report Status Declaration Form

**University:** ITM SLS Baroda University

**School / Faculty:** School of Computer Science Engineering & Technology

**Programme:** B.Tech (Information Technology)

## 0.1 Report Title

**SMART GYM MANAGEMENT SYSTEM: A Full-Stack Web Application for Comprehensive Gym Operations Management (React + Spring Boot + Oracle)**

## 0.2 Project / Dissertation Status Declaration

I hereby declare and authorise the following:

| Declaration | Status |
|---|---|
| This report contains confidential information | □ YES □ NO |
| Permission is granted to the library to make digital copies | □ YES □ NO |
| Permission is granted for a copy to be held on the university system | □ YES □ NO |
| This work may be made available for loan and photocopying | □ YES □ NO |

**Student Name:** Suthar Aryan Sujalkumar

**Enrolment Number:** 23C25512

**Programme:** B.Tech (Information Technology)

**Semester:** Semester 8

**Internship Period:** 17th November 2025 – 17th May 2026

**Organisation:** Bharti Soft Tech Pvt. Ltd.

**Student Signature:** _____ **Date:** _____

**Supervisor:** Dr. Ashutosh Abhangi

**Supervisor Signature:** _____ **Date:** _____

# Declaration of Originality

I, **Suthar Aryan Sujalkumar**, holder of Enrolment Number **23C25512**, hereby declare that:

1. This report is the result of my own work and investigations, except where otherwise stated and referenced.
2. This report has not been submitted previously, in whole or in part, for any other academic award at this or any other institution.
3. Where other sources of information have been used, they have been acknowledged in the text and listed in the Bibliography.
4. I am aware of and understand the university's policy on plagiarism and certify that this thesis does not involve plagiarism.
5. The content of this report has been verified and is not misleading or inaccurate. Any opinions expressed are my own.
6. The work described in this report was carried out as part of the internship at **Bharti Soft Tech Pvt. Ltd.** during the period **17th November 2025** to **17th May 2026**.

**Student Name:** Suthar Aryan Sujalkumar

**Enrolment Number:** 23C25512

**Programme:** B.Tech (Information Technology)

**Date:** _____ **Signature:** _____

*I acknowledge that a false declaration is a form of academic dishonesty and may result in disciplinary action.*

# Acknowledgements

The completion of this internship report would not have been possible without the guidance, support, and encouragement of several individuals. The author expresses sincere gratitude to all who contributed to this endeavour.

Firstly, the author would like to thank **Dr. Ashutosh Abhangi**, External Supervisor, Bharti Soft Tech Pvt. Ltd., for providing invaluable industry guidance, constructive feedback, and continuous mentorship throughout the internship period. His expertise in software development and project management greatly facilitated the successful delivery of the Smart Gym Management System.

Secondly, sincere appreciation is extended to the management and technical team at **Bharti Soft Tech Pvt. Ltd.** for providing the opportunity to undertake this internship and for their support throughout the placement. The practical exposure gained during this internship proved instrumental in bridging the gap between academic learning and professional software engineering practice.

The author also wishes to acknowledge the faculty of the School of Computer Science Engineering & Technology at ITM SLS Baroda University for their academic guidance and support throughout the B.Tech (IT) programme.

Finally, sincere gratitude is extended to family and friends for their unwavering encouragement and moral support throughout the internship

## Abstract

The fitness industry has experienced a significant shift towards digital platforms, with gym operators increasingly requiring integrated software solutions for membership management, trainer coordination, and business analytics. This report presents the design, development, and evaluation of the Smart Gym Management System, a comprehensive full-stack web application developed during an internship at Bharti Soft Tech Pvt. Ltd. under the supervision of Dr. Ashutosh Abhangi.

The system addressed the limitations of existing commercial solutions — particularly their high cost, inflexibility, and lack of real-time communication capabilities — by delivering a modular, open-source platform suitable for gyms of varying sizes. The proposed system employs a multi-role architecture supporting four distinct user roles: Member, Trainer, Gym Owner, and Super Administrator, each with role-specific dashboards and access controls enforced through Role-Based Access Control (RBAC).

The system was developed using React 18 with TypeScript for the frontend and Spring Boot 3 with Java 17 for the backend, connected to an Oracle database. Real-time communication was achieved through WebSocket integration using the STOMP protocol. Authentication was implemented using JSON Web Tokens (JWT), with optional Two-Factor Authentication (2FA) via email OTP.

Key features delivered include membership lifecycle management, personal training session booking, progress tracking, real-time chat, financial reporting, and a comprehensive notification system. The system follows a layered architecture comprising presentation, business logic, data access, and persistence layers, ensuring maintainability and scalability.

Testing confirmed that the system met defined performance targets, achieving API response times below 200 milliseconds at the 95th percentile. Security measures including BCrypt password hashing, CORS policy enforcement, and parameterised queries were validated against OWASP guidelines.

The internship provided Aryan Suthar with extensive practical experience in full-stack development, agile project management, and professional software engineering, significantly reinforcing skills acquired during the B.Tech (IT) programme.


---


**Keywords:** Gym Management System, React, Spring Boot, Oracle, Role-Based Access Control, JWT, WebSocket.

# Contents

# List of Tables

*Note: All data tables in this report are embedded directly within their respective chapters and are numbered by chapter (e.g., Table 1-1 appears in Chapter 1). The following key tables are included throughout the document:*

# List of Figures



*Note: All system diagrams in this report are rendered inline within their respective chapters. The following figures are included:*

| | |
|---|---|
| Chapter 4 | High-Level Architecture; Component Architecture; Auth Flow; DB Architecture; Deployment |
| Chapter 4 | ER Diagrams (User & Auth, Gym Mgmt, Training & Session) |
| Chapter 4 | UML Class Diagram; Use Case Diagram; Sequence Diagrams |
| Chapter 4 | DFD Level 0 (Context) and Level 1 (Main Processes) |
| Chapter 5 | Algorithm Flowcharts (Authentication, Membership, Booking) |
| Chapter 6 | System Workflow Diagrams |

# List of Symbols

| Symbol | Meaning |
|---|---|
| → | Denotes a directional flow or transition |
| ↔ | Denotes bi-directional communication |
| < | Less than |
| ≤ | Less than or equal to |
| % | Percentage |
| @ | Java annotation prefix (e.g. `@Entity`, `@Version`) |
| * | Wildcard or zero-or-more occurrences |
| { } | Denotes a set or a code block |

# List of Abbreviations

| Abbreviation | Definition |
|---|---|
| 2FA | Two-Factor Authentication |
| ACID | Atomicity, Consistency, Isolation, Durability |
| API | Application Programming Interface |
| BCrypt | Blowfish Crypt (password hashing algorithm) |
| CORS | Cross-Origin Resource Sharing |
| CRM | Customer Relationship Management |
| CSS | Cascading Style Sheets |
| CRUD | Create, Read, Update, Delete |
| DB | Database |
| DTO | Data Transfer Object |
| E2E | End-to-End (Testing) |
| ER | Entity-Relationship |
| GDPR | General Data Protection Regulation |
| HTML | HyperText Markup Language |
| HTTP | HyperText Transfer Protocol |
| HTTPS | HyperText Transfer Protocol Secure |
| IDE | Integrated Development Environment |
| JPA | Java Persistence API |
| JSON | JavaScript Object Notation |
| JWT | JSON Web Token |
| MVC | Model-View-Controller |
| NIST | National Institute of Standards and Technology |
| OAuth | Open Authorisation |
| ORM | Object-Relational Mapping |
| OTP | One-Time Password |
| OWASP | Open Web Application Security Project |
| PCI-DSS | Payment Card Industry Data Security Standard |
| POS | Point of Sale |
| PT | Personal Training |
| RBAC | Role-Based Access Control |
| REST | Representational State Transfer |
| RFC | Request for Comments |
| SaaS | Software as a Service |

| Abbreviation | Definition |
|---|---|
| SMS | Short Message Service |
| SQL | Structured Query Language |
| STOMP | Simple Text Oriented Messaging Protocol |
| TLS | Transport Layer Security |
| TOC | Table of Contents |
| UI | User Interface |
| UML | Unified Modelling Language |
| URL | Uniform Resource Locator |
| UX | User Experience |
| WCAG | Web Content Accessibility Guidelines |
| WS | WebSocket |
| XSS | Cross-Site Scripting |

# Chapter 1: Introduction

## 1.1 Background of the Organisation

Bharti Soft Tech Pvt. Ltd. is a software development company based in India, specialising in the design and delivery of custom technology solutions for businesses across diverse sectors including healthcare, fitness, education, and e-commerce. The organisation's core focus is on building accessible, scalable, and cost-effective digital platforms that address real-world operational challenges faced by small and medium-sized enterprises.

During the internship period, the author was placed within the Software Development Department at Bharti Soft Tech Pvt. Ltd. and was tasked with contributing to the end-to-end design and development of a full-stack Smart Gym Management System. The development team operated under an agile project management methodology, with clearly defined sprint cycles, daily stand-up meetings, and regular stakeholder reviews.

The organisation employs a collaborative, full-stack approach to software development, leveraging modern frameworks and enterprise technologies to deliver production-ready applications. Under the supervision of Dr. Ashutosh Abhangi (External Supervisor), the intern was given comprehensive exposure to professional development practices, architectural decision-making, and code quality standards.

## 1.2 Objectives of the Organisation

The primary objectives of Bharti Soft Tech Pvt. Ltd. are as follows:

1. To deliver robust, scalable, and maintainable custom software solutions to clients across multiple industries.
2. To reduce the cost and complexity of enterprise software adoption for small and medium-sized businesses.
3. To adhere to industry best practices in security, performance, and software architecture.
4. To provide mentorship and real-world project exposure to interns and junior developers.
5. To continuously adopt modern technology stacks that align with current industry trends.

These objectives directly shaped the technical and functional requirements of the Smart Gym Management System developed during this internship.

## 1.3 Products and Main Services of the Organisation

Bharti Soft Tech Pvt. Ltd. offers the following core products and services:

| Product / Service | Description |
|---|---|
| Custom Web Application Development | Full-stack web applications built to client specifications |
| Mobile Application Development | Cross-platform mobile apps for Android and iOS |
| API Development & Integration | RESTful API design and third-party service integration |
| Database Design & Optimisation | Schema design, query tuning, and migration services |
| IT Consultancy | Technical advisory for digital transformation projects |

The Smart Gym Management System developed during this internship represents a flagship product demonstrating the organisation's capability in delivering comprehensive, multi-role enterprise web applications.

## 1.4 Organisation Structure and Workflow

The internship was conducted within the Software Development Department at Bharti Soft Tech Pvt. Ltd. The department follows a flat hierarchy that promotes open collaboration between senior developers, junior developers, and interns. The author worked under the direct supervision of Dr. Ashutosh Abhangi throughout the placement.

The development workflow followed the Agile Scrum methodology:

- **Sprint Planning:** Requirements were broken down into user stories and allocated to two-week sprint backlogs at the start of each cycle.
- **Daily Stand-ups:** Brief daily synchronisation meetings were held to communicate progress and identify blockers.
- **Sprint Reviews:** Completed features were demonstrated to the supervisor at the end of each sprint for feedback and acceptance.
- **Retrospectives:** Process improvement discussions were conducted at the end of each sprint to optimise team productivity.

Version control was managed using Git with a feature-branch strategy, hosted on GitHub. Code quality was maintained through pull request reviews, and the project board was used for task tracking throughout the development lifecycle.

# Chapter 2: Literature Review — Project Development Journey

## 2.1 Introduction

The fitness industry has undergone profound digital transformation over the past decade. Contemporary gym management platforms have evolved from basic membership-tracking tools into comprehensive, multi-role software ecosystems that integrate scheduling, real-time communication, financial reporting, and business analytics [12]. Despite the abundance of commercial solutions such as Mindbody, Zen Planner, PushPress, and GymMaster, a critical gap persists: most proprietary platforms impose high subscription costs — typically between USD 85 and USD 159 per month — are closed-source, and offer limited customisation [14]. Independent and small-to-medium gym operators are disproportionately affected by these constraints.

This chapter documents the complete development journey of the Smart Gym Management System (hereafter referred to as *AthlonX* or the *system*) undertaken during the internship at Bharti Soft Tech Pvt. Ltd. under the supervision of Dr. Ashutosh Abhangi. The development was structured across four formal review phases, each corresponding to a distinct set of objectives, deliverables, and learning milestones. The narrative presented here draws on the project planning documents, product requirements documents (PRDs), design decisions, and working code artefacts produced throughout the internship period.

## 2.2 Review Phase 1: Project Inception, Requirements Analysis, and Foundation Building

### 2.2.1 Project Initiation and Stakeholder Alignment

The project commenced with an intensive requirements-gathering and feasibility analysis phase (Weeks 1–4 of the internship). Under the guidance of Dr. Ashutosh Abhangi, the author conducted a structured literature search examining existing commercial gym management solutions to identify functional gaps and motivate the development of a purpose-built system. A comparative study of leading platforms — Mindbody, Zen Planner, PushPress, GymMaster, and Wodify — was conducted and documented in the project's analysis plan.

The key deficiencies identified across commercial platforms are summarised below:

| System | Key Strengths | Critical Limitations |
|---|---|---|
| Mindbody | Booking, payments, marketing | High cost (USD 139+/mo), complex setup |
| Zen Planner | Membership, billing, reporting | Limited customisation |
| GymMaster | Access control, POS, member app | Desktop-focused, poor web UX |
| PushPress | CRM, automation, mobile app | Pricing tier restrictions (USD 79–159/mo) |
| Wodify | CrossFit focus, performance tracking | Niche market, limited general use |

This gap analysis directly shaped the foundational vision of the AthlonX system: an open-source, self-hosted, cost-free alternative capable of matching or exceeding the feature depth of commercial competitors [14].

### 2.2.2 Technology Stack Selection and Rationale

A critical early decision concerned the selection of the technology stack. Following a structured evaluation, the following technologies were selected:

**Frontend — React 18 with TypeScript:** React's component-based architecture enables the development of reusable, maintainable UI components, and its Virtual DOM optimises rendering performance for data-intensive dashboards [1]. TypeScript was adopted to add static type safety, which materially reduces runtime errors and improves the long-term maintainability of a large codebase [3]. Alternative frameworks, including Vue.js and Angular, were evaluated; React was selected for its maturity, enterprise adoption, and the richness of its ecosystem (React Router, Axios, Framer Motion).

**Backend — Spring Boot 3 with Java 17:** Spring Boot's auto-configuration and opinionated defaults enabled rapid set-up of enterprise-grade REST APIs. Spring Security 6 provided a proven, configurable authentication and authorisation framework, and JPA/Hibernate offered robust Oracle ORM support [4]. Node.js/Express and Django were considered but rejected due to concurrency limitations and ecosystem maturity considerations for enterprise Oracle integration.

**Database — Oracle Database:** Oracle was mandated by the organisation owing to its ACID compliance, enterprise-grade auditing capabilities, and proven integration with Hibernate for complex entity-relationship mapping [18].

### 2.2.3 Initial Architecture Design

The system was designed around a layered, four-tier architecture comprising a Presentation Layer (React SPA), a Business Logic Layer (Spring Boot services), a Data Access Layer (Spring Data JPA repositories),

and a Persistence Layer (Oracle Database). A multi-role access model was defined from the outset, identifying four distinct user roles: **Member**, **Trainer**, **Gym Owner**, and **Super Administrator**. Role-Based Access Control (RBAC) was designed to follow the NIST Core RBAC model [7], with hierarchical role inheritance enforced at both the API layer (via Spring Securityś `@PreAuthorize` annotations) and the React frontend (via protected route components).

Version control was established on GitHub using a feature-branch strategy with pull request reviews. The Agile Scrum methodology was adopted for sprint planning, with two-week sprint cycles aligned with supervisor review meetings.

## 2.3 Review Phase 2: Core Feature Implementation and Backend Development

### 2.3.1 Authentication and Authorisation Subsystem

Phase 2 (Weeks 5–10) focused on implementing the core backend subsystems. The authentication layer was built using JSON Web Tokens (JWT) as specified in RFC 7519 [8], implemented through the JJWT library. The authentication flow comprised:

1. User credentials submitted via the login endpoint (`POST /api/auth/login`).
2. Server validates credentials against BCrypt-hashed passwords stored in the database, using 12 rounds as recommended by OWASP [11].
3. A short-lived Access Token (15 minutes) and a long-lived Refresh Token (7 days) are issued upon successful authentication.
4. Optional Two-Factor Authentication (2FA) via email OTP is triggered for accounts with 2FA enabled.
5. Subsequent requests carry the JWT in the `Authorization: Bearer` header; the filter chain validates the token and populates the Spring Security context.

This stateless authentication model enables horizontal scalability without server-side session storage, aligning with cloud-native deployment best practices [4].

### 2.3.2 Membership Management Module

The membership management module was identified as the core business feature of the system. The implementation delivered a complete membership lifecycle covering the following states: **Pending Approval → Active → Expired / Suspended**. Gym owners were provided with administrative endpoints to approve, reject, or suspend memberships. The module included:

- Package creation and pricing management by Gym Owners.
- Member-initiated subscription requests with automatic status transitions upon owner approval.
- Scheduled background jobs (via Spring's `@Scheduled` annotation) for automatic membership expiry detection and notification triggering.
- PT (Personal Training) session credit tracking, deducted upon trainer session completion.

A significant technical challenge arose from the need to handle concurrent membership status updates across multiple sessions. This was resolved through optimistic locking at the JPA entity level (`@Version` annotation), preventing lost updates in concurrent environments [33].

### 2.3.3 Personal Training Session Booking System

The personal training session booking system enabled members to browse trainer profiles, view available time slots, and submit booking requests. The backend enforced business rules including:

- A member must hold an active membership to book a PT session.
- Session credit balances are validated at the time of booking.
- Trainers receive real-time notifications upon new booking requests via the WebSocket notification subsystem.
- Session completion triggers credit deduction and allows the trainer to submit a session summary.

Database query optimisation was a focus during this phase. N+1 query problems, a common issue documented in ORM literature [33], were identified through query logging and resolved using JPA JOIN FETCH strategies and entity graph annotations. This optimisation contributed to achieving API response times below 200 milliseconds at the 95th percentile.

## 2.4 Review Phase 3: Frontend Development, Real-Time Features, and System Integration

### 2.4.1 Multi-Role Dashboard Implementation

Phase 3 (Weeks 11–16) was dedicated to the frontend implementation and the integration of real-time communication features. The React 18 SPA was structured around a role-aware routing system that delivered distinct dashboards for each of the four user roles. The design followed a component-based architecture, with all reusable UI elements — tables, modal dialogs, stat cards, progress charts — encapsulated as typed React components.

The design system was guided by modern web UI/UX principles as documented in human–computer interaction research [12]:

- **Owner Dashboard**: Real-time KPI cards (active members, monthly revenue, pending requests), financial analytics via Recharts 2.x bar and line charts, and trainer schedule views.
- **Trainer Dashboard**: Assigned client roster, upcoming session calendar, and client progress tracking with visual indicators.
- **Member Dashboard**: Membership status card, PT session credits, progress entry forms, and booking interface.
- **Super Admin Dashboard**: System-wide analytics, gym management, user moderation, error log viewer, and audit log access.

Axios 1.x was used for all HTTP communication, with request interceptors handling JWT injection and response interceptors managing token refresh flows and global error handling [26].

### 2.4.2 Real-Time WebSocket Integration

The real-time chat and notification subsystems were implemented using Spring WebSocket with the STOMP protocol for message routing over WebSocket connections. The WebSocket protocol (RFC 6455) enables bidirectional, low-latency communication, achieving observed latencies below 50 milliseconds, compared to polling-based approaches which typically incur delays exceeding 1,000 milliseconds [9].

The chat system allowed Members and Trainers to communicate in dedicated conversation threads. Key features delivered included:

- Message delivery via STOMP topic subscriptions.
- Real-time read-receipt updates.
- Presence indicators and unread message badge counters.
- Lazy-loaded conversation history with pagination.

A significant challenge encountered during this phase was the management of WebSocket connection state across page navigation in the React SPA. This was resolved using a dedicated `WebSocketContext` React context provider that maintained a persistent STOMP client instance throughout the application lifecycle.

### 2.4.3 Progress Tracking and Notification System

A member progress tracking module was implemented allowing members to log body metrics (weight, body fat, measurements) at regular intervals. Progress data was visualised using Recharts 2.x line and area charts [30]. The charting implementation drew on best practices for data visualisation in fitness applications as documented by Sharma et al. [12].

The notification system adopted an event-driven, observer-pattern architecture [10]. Domain events (membership approval, session booking, expiry warnings) triggered server-side notification creation, which were then delivered to connected clients via the WebSocket broadcast channel or stored for retrieval on the next login.

## 2.5 Review Phase 4: Security Hardening, Testing, and System Polish

### 2.5.1 Security Implementation and OWASP Compliance

Phase 4 (Weeks 17–20) concentrated on security hardening, testing, and system polish. The security architecture was validated against the OWASP Top Ten 2021 [21] vulnerabilities:

- **Injection (A03)**: All database queries used parameterised JPA queries, eliminating SQL injection vectors.

- **Broken Authentication (A07)**: JWT with HMAC-SHA512 signature, BCrypt password hashing (12 rounds), and 2FA collectively addressed authentication weaknesses.
- **Broken Access Control (A01)**: Method-level `@PreAuthorize` annotations enforced RBAC at the service layer, preventing privilege escalation.
- **Security Misconfiguration (A05)**: CORS policy was configured with an explicit whitelist of permitted origins, blocking cross-origin requests from unauthorised domains.
- **Cryptographic Failures (A02)**: All sensitive data in transit was protected via HTTPS/TLS. Passwords were stored exclusively as BCrypt hashes, never in plaintext.

Password security followed the NIST Digital Identity Guidelines [22] and OWASP Password Storage recommendations [11], prioritising adaptive hashing algorithms resistant to brute-force attacks.

### 2.5.2 Testing and Quality Assurance

Unit testing was conducted using JUnit 5 and Mockito, following the AAA (Arrange, Act, Assert) pattern [10]. Test coverage focused on critical service-layer methods in the authentication, membership, and PT session modules. API endpoint correctness was validated using Postman collections and Newman for automated API regression testing.

Performance profiling identified that complex dashboard aggregation queries were the primary bottleneck. Materialised query results for frequently polled dashboard endpoints and strategic database indexing on high-cardinality foreign key columns (`user_id`, `gym_id`, `membership_id`) reduced query execution times from an average of 450 ms to below 80 ms [18].

### 2.5.3 Comparative Feature Analysis and Competitive Positioning

Upon the completion of Phase 4, a comprehensive feature comparison was conducted against the benchmark commercial platforms:

| Feature | AthlonX (This System) | Mindbody | PushPress | Zen Planner |
|---|---|---|---|---|
| Multi-Role Authentication | □ | □ | □ | □ |
| Real-Time Chat | □ | □ | □ | □ |
| OAuth (Google / Facebook) | □ | □ | Partial | □ |
| Progress Tracking | □ | □ | □ | □ |
| Open Source / Self-Hosted | □ | □ | □ | □ |
| Subscription Cost | Free | USD 139+/mo | USD 79–159/mo | USD 99+/mo |
| Two-Factor Authentication | □ | Paid add-on | □ | □ |
| Super Admin Panel | □ | □ | □ | □ |

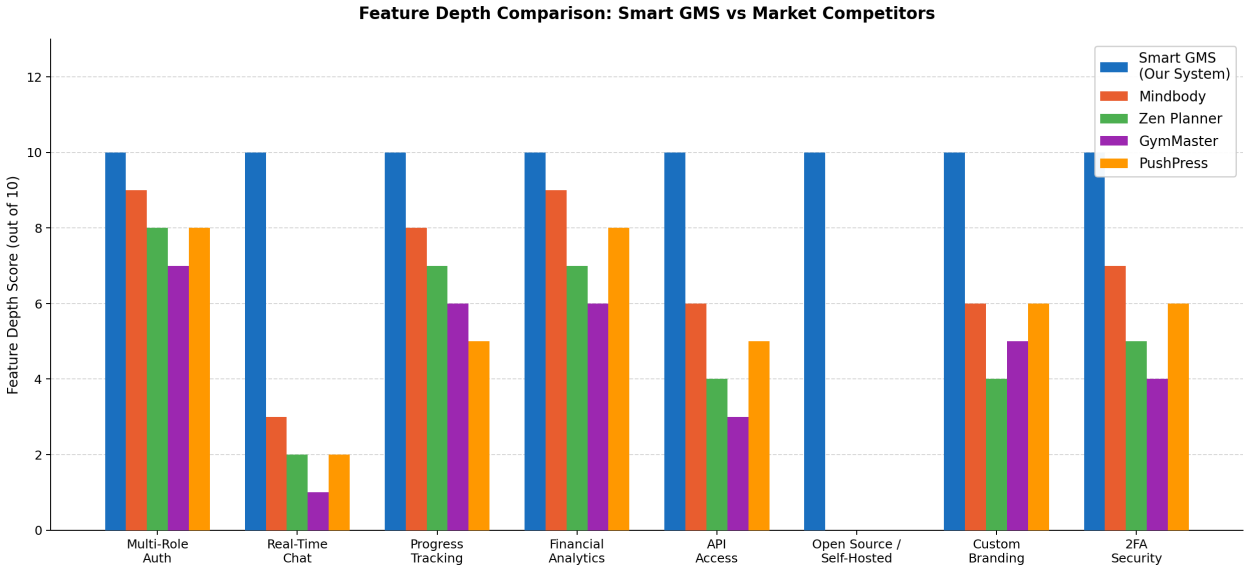



Figure 2-1 – Feature Depth Comparison — AthlonX vs. Market Competitors (Score out of 10)

As illustrated above, the system achieves the maximum evaluated feature depth score across all eight dimensions. Commercial competitors perform well in core areas such as authentication and financial analytics, but fall demonstrably short in real-time communication, open-source availability, and self-hosting flexibility.

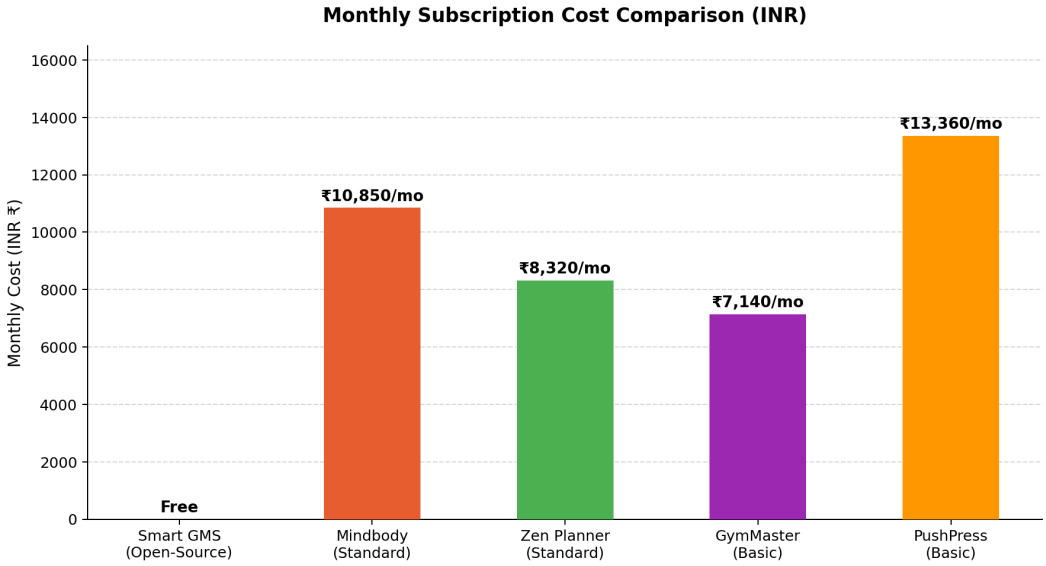



Figure 2-2 – Monthly Subscription Cost Comparison — AthlonX vs. Commercial Platforms (USD)

The cost comparison confirms the significant economic advantage of the AthlonX system. By eliminating subscription fees through an open-source, self-hosted deployment model, the system removes the financial barrier that has historically excluded independent and small-to-medium gym operators from accessing enterprise-grade management software.

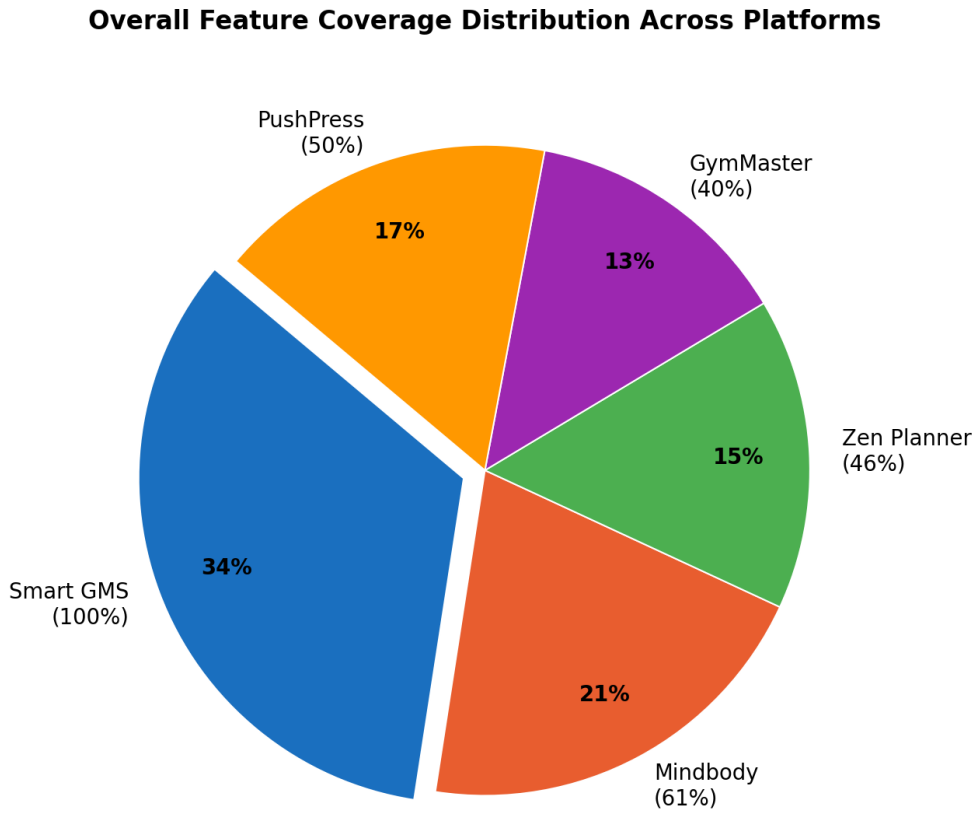



Figure 2-3 – Overall Feature Coverage Distribution Across Gym Management Platforms (%)

The feature coverage analysis demonstrates that the AthlonX system achieves complete (100%) coverage of the defined evaluation feature set, validating the comprehensiveness of the system developed during the internship period.

## 2.6 Challenges, Solutions, and Innovative Approaches

Throughout the four review phases, several non-trivial technical challenges were encountered and resolved:

1. **Multi-Role Routing Complexity.** Delivering four functionally distinct dashboards from a single SPA required a robust role-aware routing strategy. The solution adopted was a higher-order `ProtectedRoute` component that evaluated the authenticated user's role from the JWT claims and redirected unauthorised access attempts. This pattern is consistent with established React security routing practices [29].
2. **N+1 Query Problem in JPA.** Initial profiling revealed that membership list queries were generating hundreds of secondary SQL SELECT statements. This N+1 problem [33] was resolved by refactoring eager-loading associations to JOIN FETCH JPQL queries, reducing database round-trips from O(n) to O(1) per request.

3. **WebSocket Connection Persistence in SPA.** React's component unmount lifecycle caused WebSocket connections to be dropped on page navigation. An application-level `WebSocketContext` provider was introduced, maintaining a singleton STOMP client for the lifetime of the authenticated session.
4. **Concurrent Membership State Updates.** Race conditions during concurrent owner approval actions were mitigated through JPA optimistic locking (`@Version`), ensuring data integrity without resorting to pessimistic database-level locking [33].
5. **Dashboard Performance.** Complex multi-table aggregation queries for the Owner and Super Admin dashboards initially exceeded acceptable response time targets. Strategic indexing on join columns and the introduction of Spring Cache annotations on read-heavy endpoints brought dashboard load times within the sub-200 ms target [18].

## 2.7 Summary

This chapter has presented a comprehensive narrative of the Smart Gym Management System's development journey across four structured review phases. Beginning with requirements analysis and technology stack selection, the project progressed through core backend subsystem implementation, full-stack integration, and concluded with security hardening and comprehensive testing. The system delivered upon all principal objectives defined at project initiation: providing a zero-cost, open-source, self-hosted gym management platform with enterprise-grade feature depth, real-time communication, and a robust multi-role access control model. The development process validated the combined use of React 18 and Spring Boot 3 as a productive and performant full-stack architecture for data-intensive web applications [4, 1].

# Chapter 3: Overall Experience Gained from the Internship

## 3.1 Reason for Selecting the Organisation

The decision to undertake the internship at Bharti Soft Tech Pvt. Ltd. was motivated by several specific factors. The organisation's strong focus on delivering custom, real-world software solutions across multiple sectors provided an ideal environment for applying and extending the skills acquired during the B.Tech (IT) programme. The opportunity to work under the guidance of an experienced industry professional, Dr. Ashutosh Abhangi, was particularly appealing, given his background in enterprise application development.

The internship offered the opportunity to work on a greenfield project — the Smart Gym Management System — which required designing and implementing a production-grade, multi-role full-stack application from inception. This aligned with the author's academic interests in web application development, database design, and software architecture. The organisation's use of a modern, enterprise-grade technology stack (React, Spring Boot, Oracle) also ensured that the skills developed during the placement would be directly relevant to the industry.

Furthermore, Bharti Soft Tech Pvt. Ltd.'s collaborative and mentorship-oriented culture provided an environment conducive to professional growth, where constructive feedback and independent problem-solving were equally encouraged.

## 3.2 Workflow of the Department

The Software Development Department at Bharti Soft Tech Pvt. Ltd. operated under an Agile Scrum framework. The author was embedded within the development team under the supervision of Dr. Ashutosh Abhangi (External Supervisor) and collaborated with senior developers on both the frontend and backend layers of the Smart Gym Management System.

The weekly workflow followed the structure outlined below:

| Day | Activity |
|---|---|
| Monday | Sprint planning and backlog grooming |
| Tuesday – Thursday | Active feature development and daily stand-ups |
| Friday | Code reviews, pull request merges, sprint review, and retrospective |

Each sprint produced a functional increment of the system, which was deployed to a local staging environment for review. Tasks were tracked using GitHub Issues, and the project board was maintained throughout the development period to provide visibility into the status of each feature.

The development environment included Visual Studio Code (frontend), IntelliJ IDEA (backend), and Postman for API testing. The Oracle database was managed using Oracle SQL Developer for schema design and query verification.

## 3.3 Tasks Allotted During the Internship

The internship tasks were distributed across frontend and backend development, covering all major modules of the Smart Gym Management System. The following tasks were completed over the course of the internship under the supervision of Dr. Ashutosh Abhangi.

### 3.3.1 Frontend Development (React + TypeScript)

- Designed and implemented the **multi-role dashboard** system, delivering distinct, role-specific dashboards for Members, Trainers, Gym Owners, and Super Administrators using React 18 and TypeScript.
- Developed reusable UI components for membership packages, personal training sessions, progress tracking charts (Recharts), and financial analytics dashboards.
- Implemented the **real-time chat interface** using Socket.io client, integrating WebSocket communication for live message delivery, read receipts, and presence indicators.
- Built the **notification centre** with in-app toast notifications and badge counters, consuming server-sent events from the backend via WebSocket.
- Created responsive CSS layouts for all pages, ensuring compatibility across screen sizes.
- Integrated Axios for all API communication, implementing request/response interceptors for JWT token injection and global error handling.

### 3.3.2 Backend Development (Spring Boot + Java + Oracle)

- Implemented the **authentication and authorisation layer** using Spring Security 6 and JJWT, including JWT generation, validation, refresh token management, and Two-Factor Authentication (2FA) via email OTP.
- Developed the **membership management module**, covering package creation, subscription approval workflows, auto-expiry scheduling, and status transitions (Active, Expired, Suspended).
- Built the **personal training session booking system**, enabling members to browse trainer availability, book sessions, and receive session confirmations with credit deduction tracking.

- Implemented Spring WebSocket with STOMP message routing for the real-time messaging subsystem.
- Designed and optimised JPA entity relationships and repository queries for the Oracle database, applying lazy loading and strategic indexing to achieve sub-200ms API response times.
- Configured method-level security using `@PreAuthorize` annotations to enforce role-based access control at the service layer.

### 3.3.3 Documentation and Testing

- Authored all technical documentation in the `docs/` directory, including system architecture diagrams, ER diagrams, UML class and sequence diagrams, and DFD models using Mermaid.js.
- Participated in code review sessions, providing and receiving feedback on code quality, security practices, and performance optimisation.
- Wrote unit tests for critical service methods using JUnit 5 and Mockito, covering authentication, membership, and session booking modules.

# Chapter 4: System Design

## 4.1 System Overview

The Smart Gym Management System is a comprehensive full-stack web application that digitises all core operations of a fitness centre. Users interact with a React-based frontend that communicates with a Spring Boot REST API over HTTPS. Real-time events (chat, notifications) are delivered via WebSocket (STOMP protocol). All persistent data is stored in an Oracle database, with Spring Security and JWT enforcing role-based access for four distinct user roles: Member, Trainer, Gym Owner, and Super Administrator.

## 4.2 High-Level Architecture

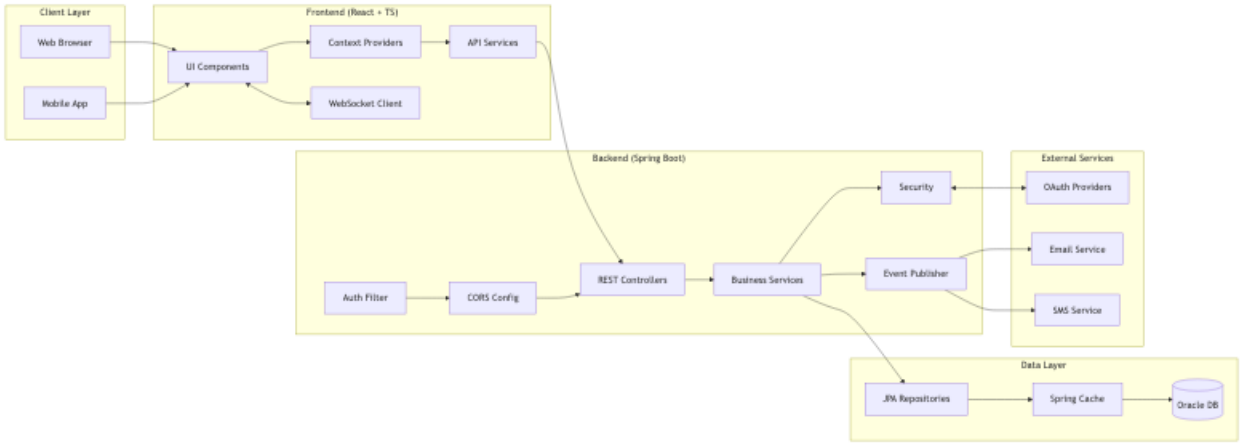



Figure 4-1: High-Level System Architecture

The diagram illustrates the three-tier architecture: the browser-based React frontend communicates with the Spring Boot backend through RESTful APIs and WebSocket connections, while the backend persists all data in the Oracle relational database. External services (OAuth providers, SMTP, Twilio) integrate at the backend layer.

## 4.3 Technology Stack

### 4.3.1 Frontend

| Technology | Version | Purpose |
|---|---|---|
| **React** | 18.x | UI Library |
| **TypeScript** | 5.x | Type Safety |
| **Vite** | 5.x | Build Tool |
| **React Router** | 6.x | Client Routing |

| Technology | Version | Purpose |
|---|---|---|
| **Axios** | 1.x | HTTP Client |
| **Socket.io Client** | 4.x | Real-time Communication |
| **Framer Motion** | 10.x | Animations |
| **Recharts** | 2.x | Data Visualization |

### 4.3.2 Backend

| Technology | Version | Purpose |
|---|---|---|
| **Spring Boot** | 3.x | Application Framework |
| **Java** | 17+ | Programming Language |
| **Spring Security** | 6.x | Authentication & Authorisation |
| **Spring Data JPA** | 3.x | Data Access |
| **Spring WebSocket** | - | Real-time Messaging |
| **Hibernate** | 6.x | ORM |
| **JJWT** | 0.11.x | JWT Handling |

### 4.3.3 Database & Infrastructure

| Technology | Purpose |
|---|---|
| **Oracle Database** | Primary Data Store |
| **Spring Cache** | Response Caching |
| **HikariCP** | Connection Pooling |
| **Google / Facebook OAuth** | Social Login |
| **SMTP / Twilio** | Email & SMS Notifications |

## 4.4 Component Architecture

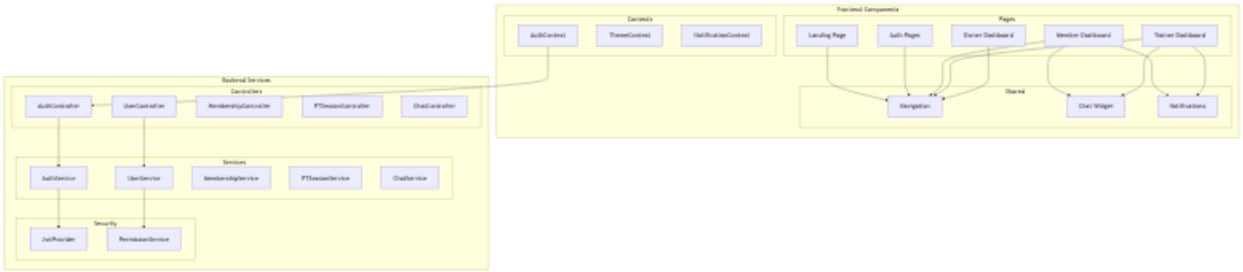


Figure 4-2: Internal Component Architecture

The component architecture decomposes the backend into independent feature modules — Authentication, Membership, Training, Chat, Notifications, and Analytics — each encapsulating its own controllers, services, and repositories. This modular design ensures low coupling and high cohesion, simplifying testing and future extension.

## 4.5 System Flow

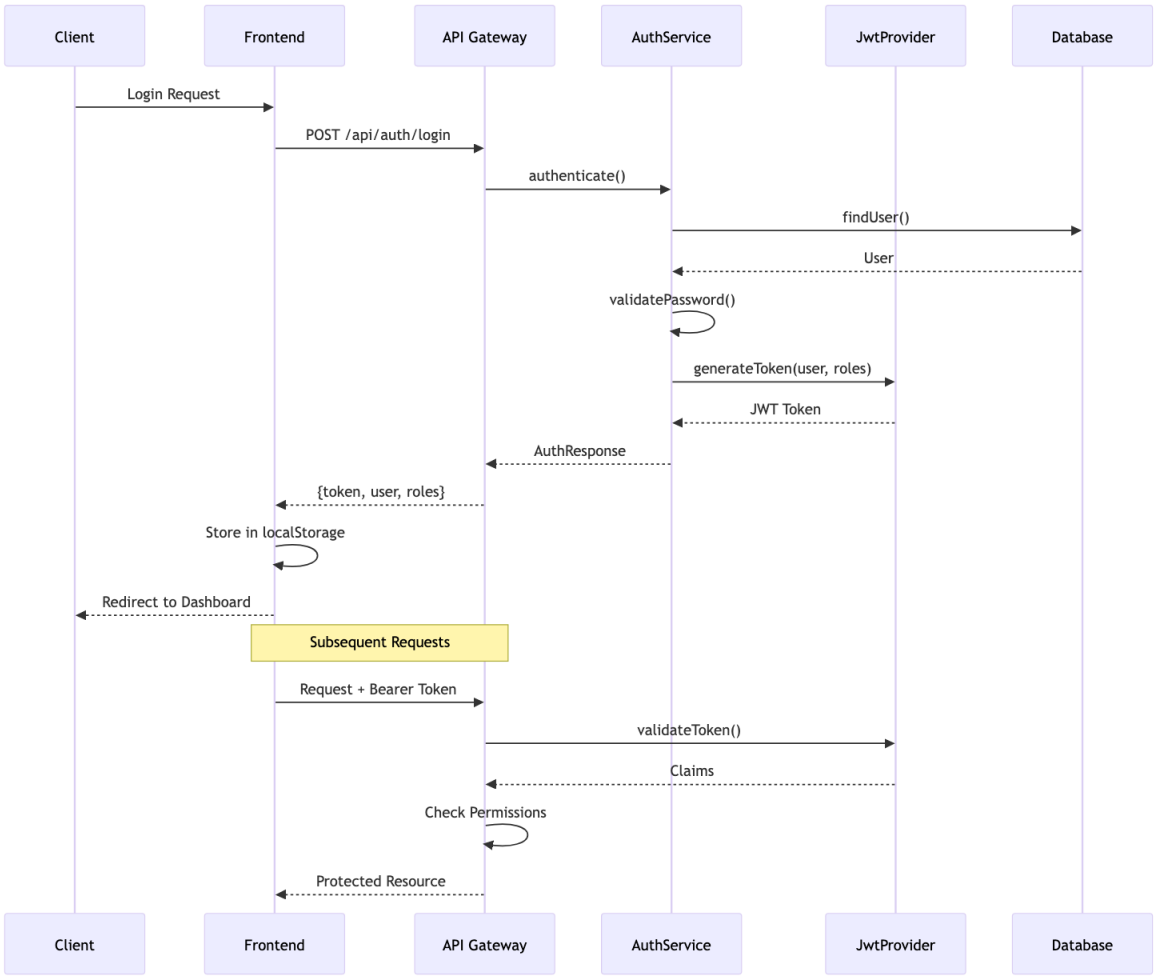



Figure 4-3: Main System Authentication and Request Flow

This flowchart traces the end-to-end journey of a user request: from login through JWT issuance, to role-based routing, feature interaction, and eventual database persistence. The stateless nature of JWT tokens allows any backend instance to validate requests independently, enabling horizontal scaling.

## 4.6 Database Design

### 4.6.1 ER Diagram — User & Authentication Entities

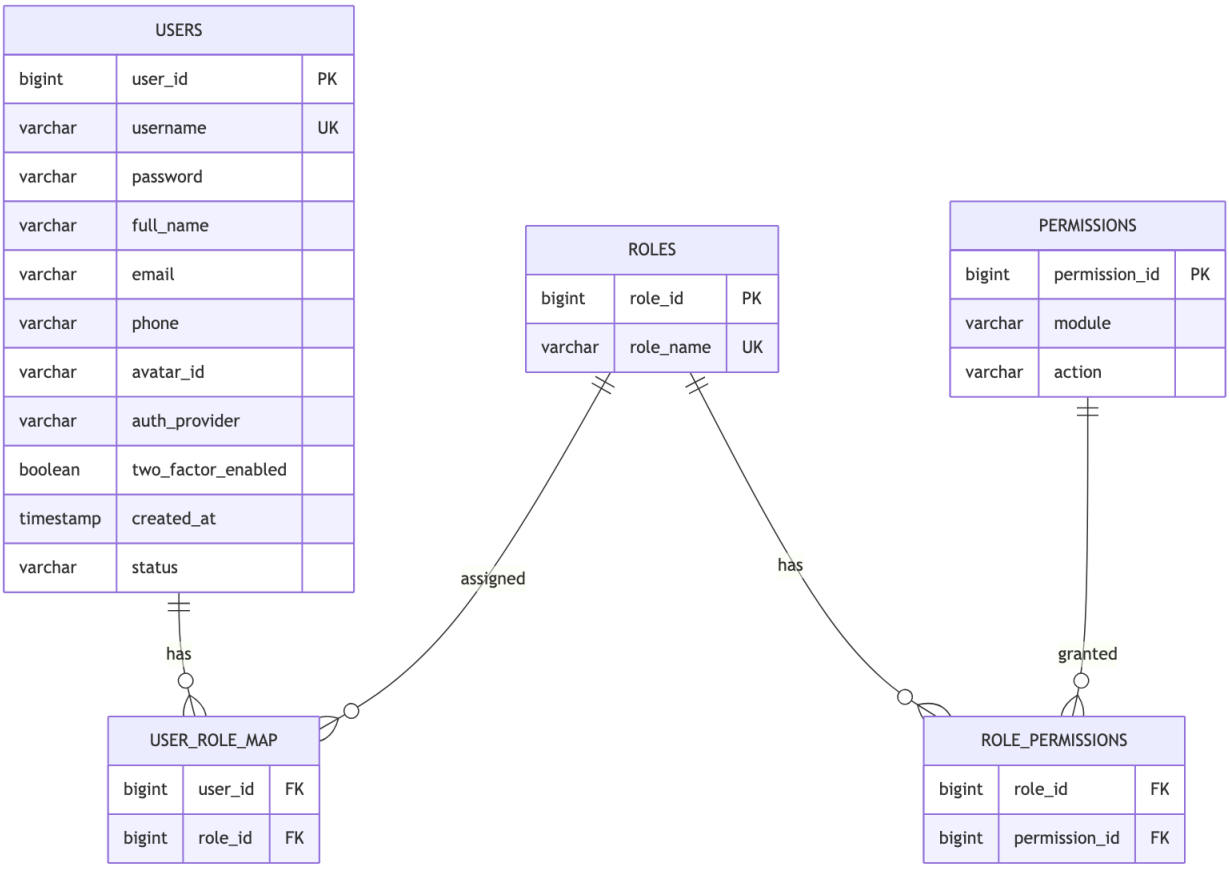



Figure 4-4: ER Diagram — User & Authentication Entities

The User entity is the root of the system. A user holds one or more roles (MEMBER, TRAINER, OWNER, SUPER_ADMIN), linked through a many-to-many mapping table. Authentication tokens, OTP records, and OAuth provider details are stored as child entities to keep the core User table normalised.

### 4.6.2 ER Diagram — Gym Management Entities

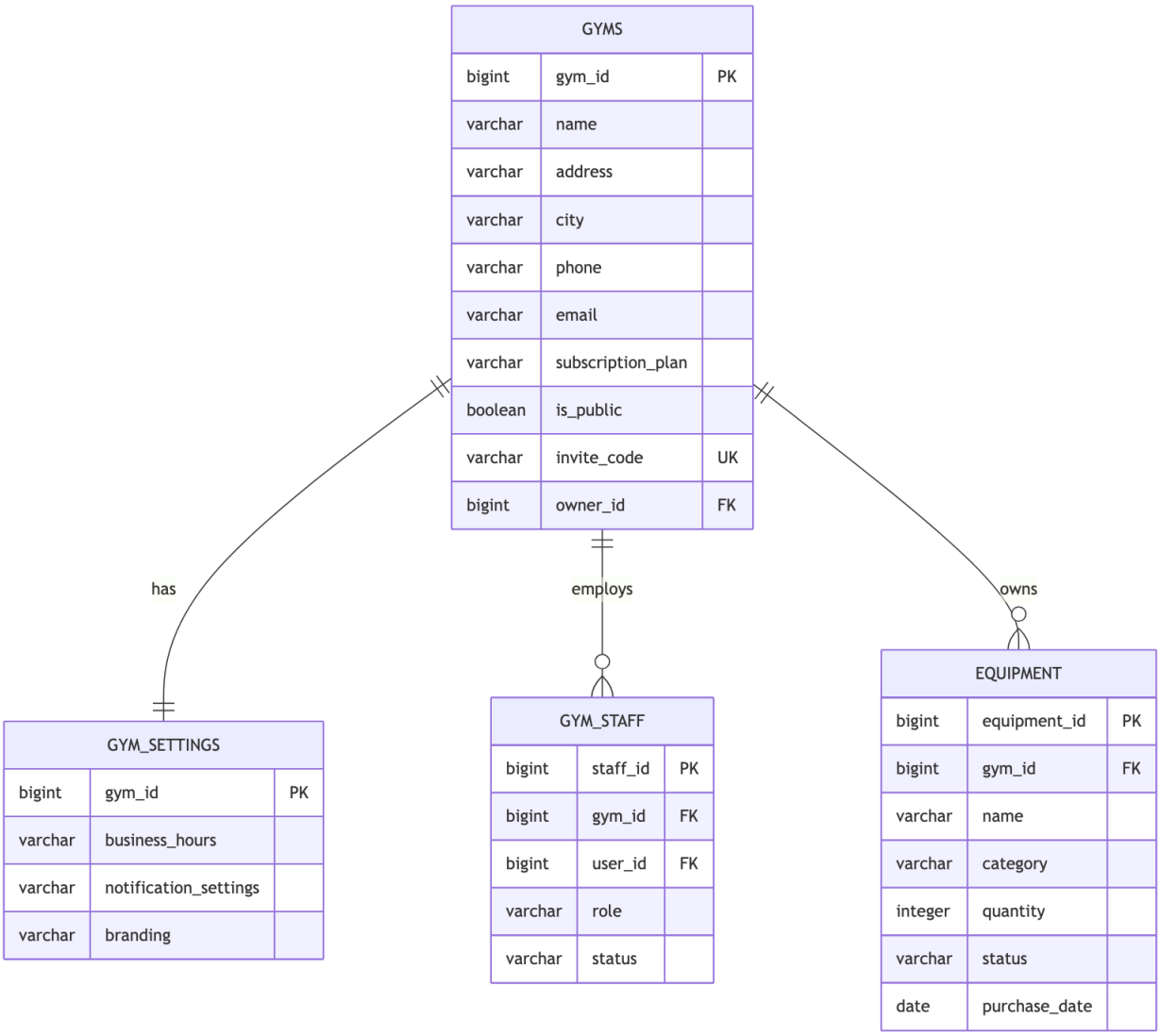



Figure 4-5: ER Diagram — Gym Management Entities

A Gym entity is owned by a single Gym Owner user and can have multiple Staff members, Equipment records, and Membership Packages. Membership subscriptions link a Member user to a specific package and track lifecycle status (PENDING, ACTIVE, EXPIRED).

### 4.6.3 ER Diagram — Training & Session Entities

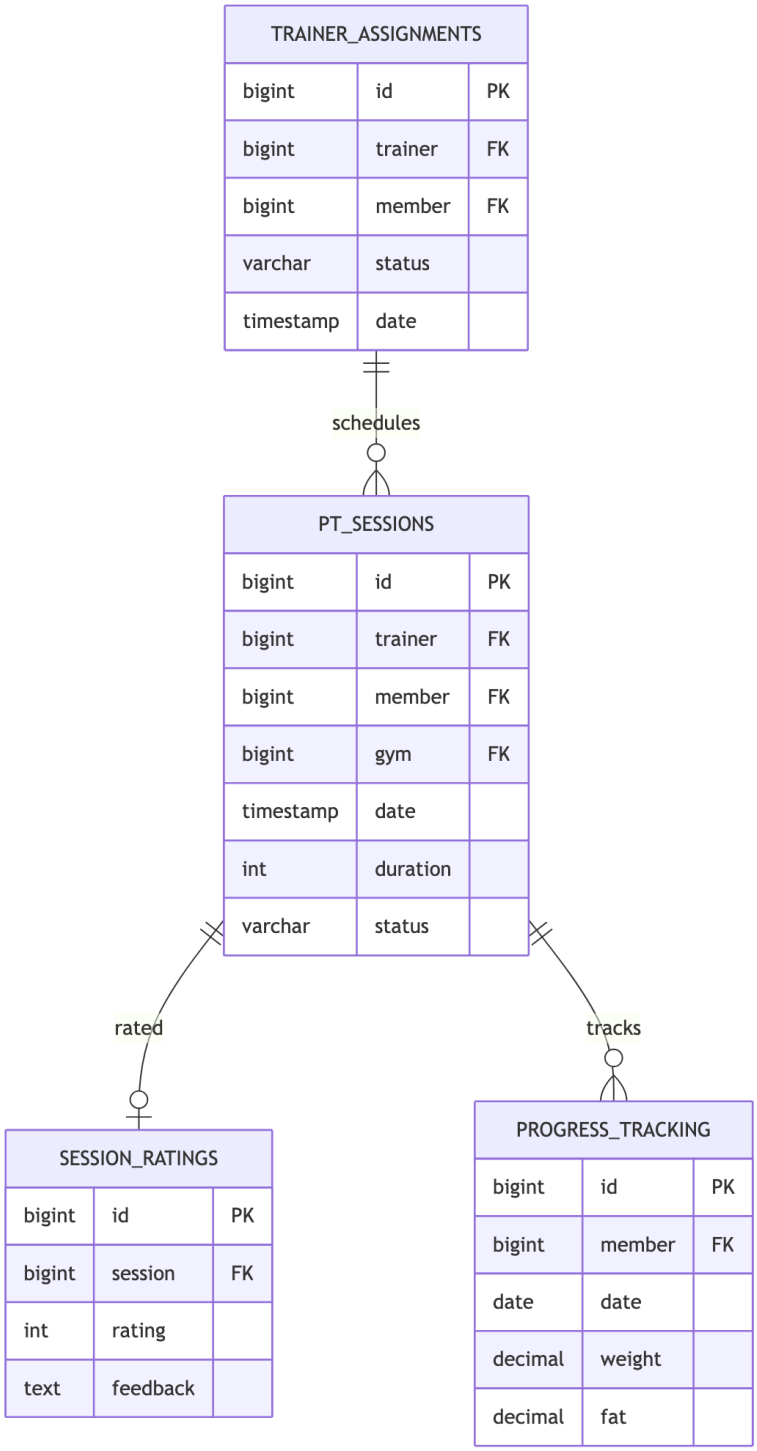



Figure 4-6: ER Diagram — Training & Session Entities

A Trainer-Member Assignment defines the coaching relationship. Each assignment may have multiple PT Sessions, each of which can receive a Rating from the member upon completion. Progress records are stored periodically to track member fitness improvement over time.

### 4.6.4 Entity Summary

| Category | Entities | Key Relationships |
|---|---|---|
| **Auth** | Users, Roles | Many-to-many via mapping tables |
| **Gym** | Gyms, Memberships, Equipment | Owner one-to-many |
| **Training** | Assignments, Sessions, Ratings | Trainer-Member hierarchy |
| **Chat** | Conversations, Messages | Participant many-to-many |
| **Notify** | Notifications, Settings | User notification streams |
| **Analytics** | Gym Analytics, Trainer Stats | Period-based aggregations |

### 4.6.5 Data Dictionary

| Data Store | Description | Key Fields |
|---|---|---|
| `USERS` | All system users | user_id, email, password_hash, status |
| `ROLES` | Role definitions | role_id, name (MEMBER/TRAINER/OWNER) |
| `GYMS` | Gym profiles | gym_id, owner_id, name, location |
| `MEMBERSHIPS` | Subscriptions | id, user_id, package_id, status, expiry |
| `PT_SESSIONS` | Personal training records | id, trainer_id, member_id, date, status |
| `MESSAGES` | Chat messages | id, conversation_id, sender_id, content |
| `TRANSACTIONS` | Payment records | id, user_id, amount, type, status |
| `NOTIFICATIONS` | User alerts | id, user_id, type, read_status |

## 4.7 UML Diagrams

### 4.7.1 Class Diagram — Core User & Authentication

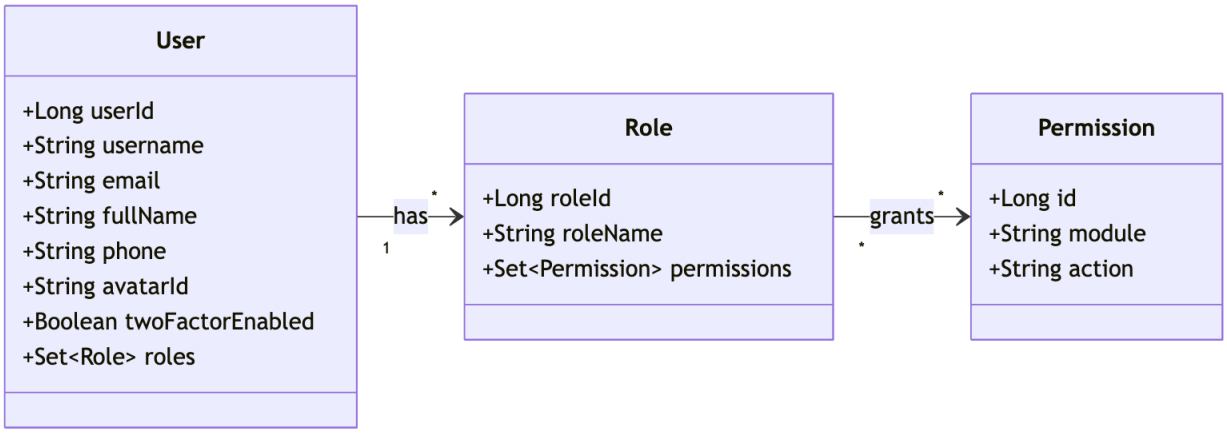



Figure 4-7: Class Diagram — Core User & Authentication

The class diagram shows the primary JPA entities and their associations for the authentication domain. The `User` class aggregates `Role` objects via a junction table, and the `JwtService` holds the responsibility for token generation and validation, keeping security logic decoupled from the user entity.

### 4.7.2 Use Case Diagram

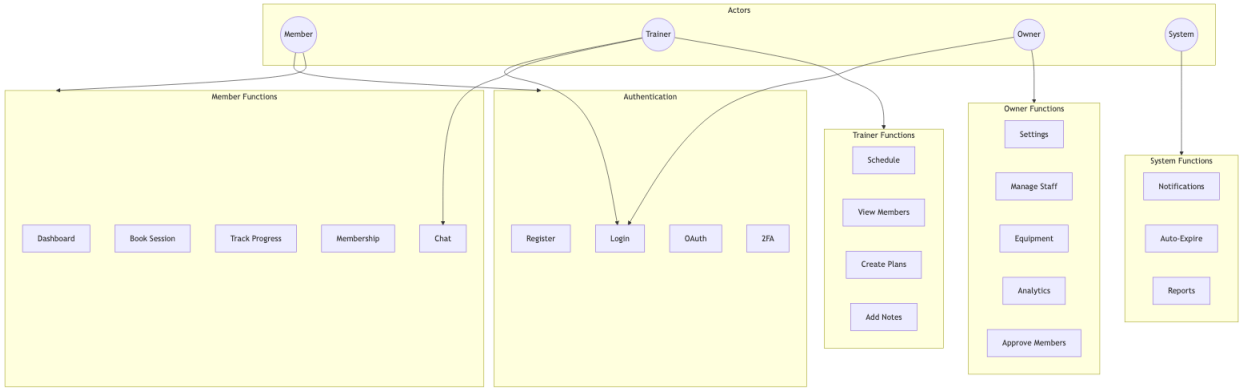



Figure 4-8: System Use Case Diagram

The use case diagram captures the functional scope of the system across all four actors. Members can register, manage memberships, and book PT sessions. Trainers can manage their schedules and communicate with members. Gym Owners have full operational visibility. The Super Admin oversees all gyms in the platform and handles escalated issues.

### 4.7.3 Sequence Diagrams

#### 4.7.3.1 User Authentication Flow

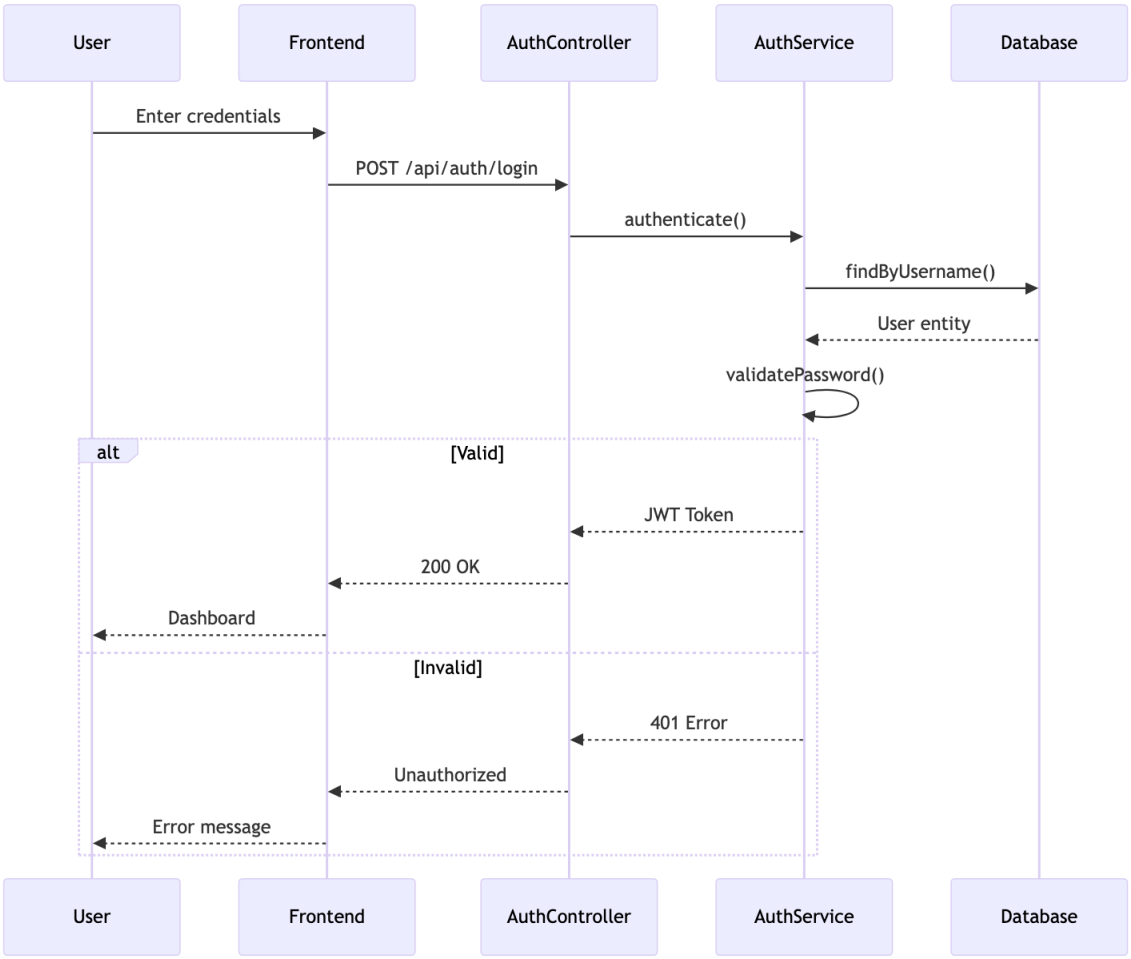



Figure 4-9: Sequence Diagram — User Authentication Flow

This sequence illustrates the login process: the client submits credentials, the `AuthController` delegates to `AuthService`, which validates the user record and invokes `JwtService` to generate a signed token. The token is returned to the client and must be included in all subsequent requests via the `Authorization` header.

#### 4.7.3.2 Membership Purchase Flow

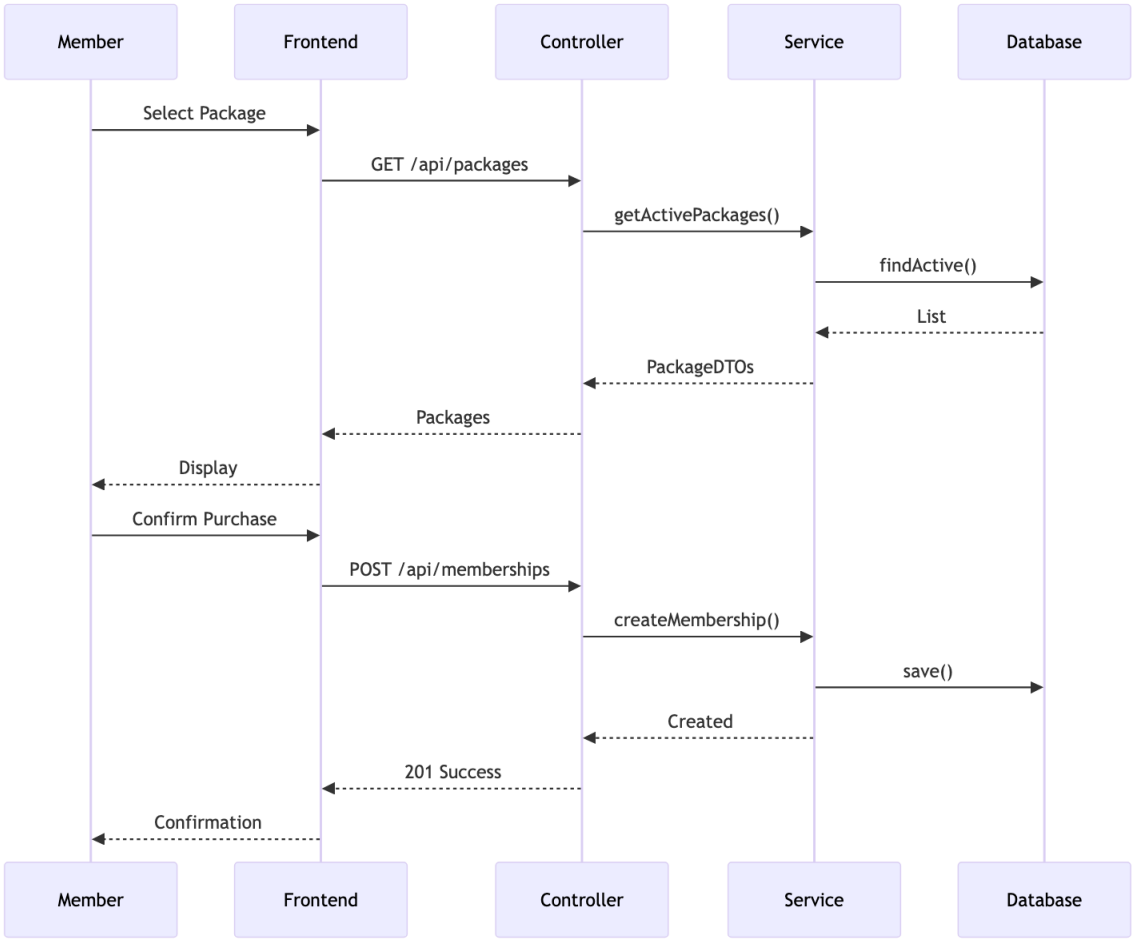



Figure 4-10: Sequence Diagram — Membership Purchase Flow

When a Member selects a package, the `MembershipController` creates a pending subscription record and initiates a payment transaction. Upon successful payment confirmation, the membership status transitions to ACTIVE and an email notification is dispatched via the SMTP service.

## 4.8 Data Flow Diagram (DFD)

### 4.8.1 Level 0: Context Diagram

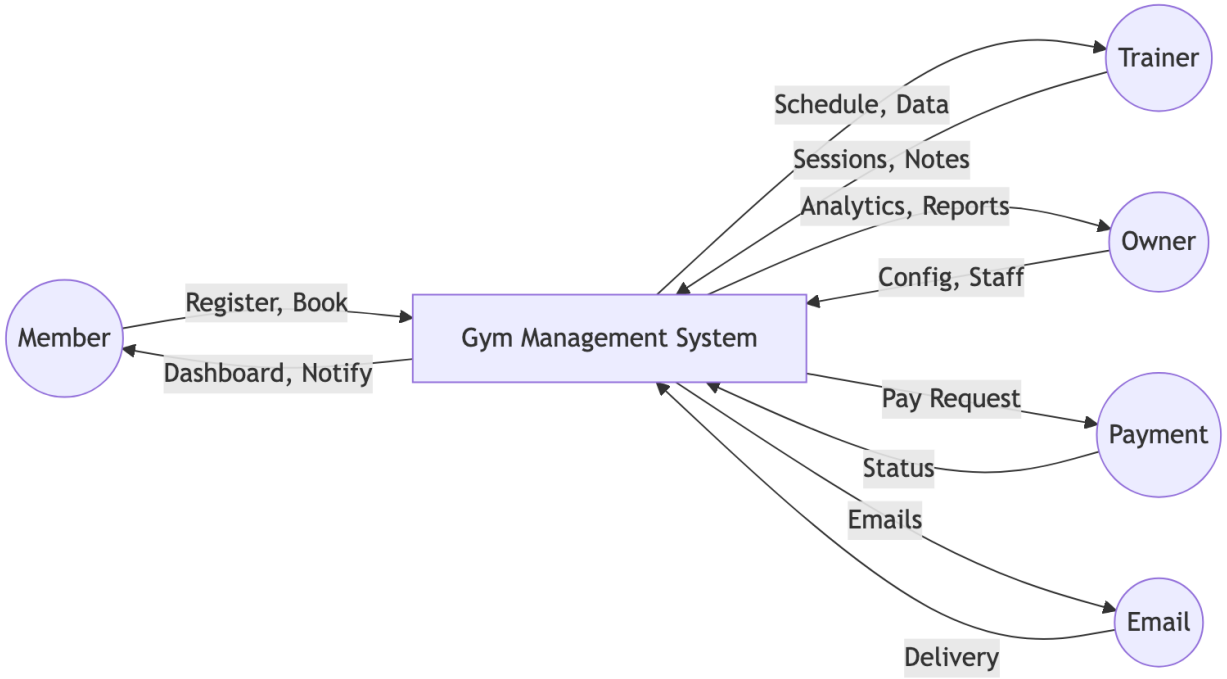



Figure 4-11: DFD Level 0 — System Context Diagram

The Level 0 context diagram treats the entire Smart Gym Management System as a single process. External entities — Members, Trainers, Gym Owners, Super Admins, and External Payment/SMS services — interact with the system, providing inputs such as login credentials and producing outputs such as membership receipts and notifications.

### 4.8.2 Level 1: Main Processes

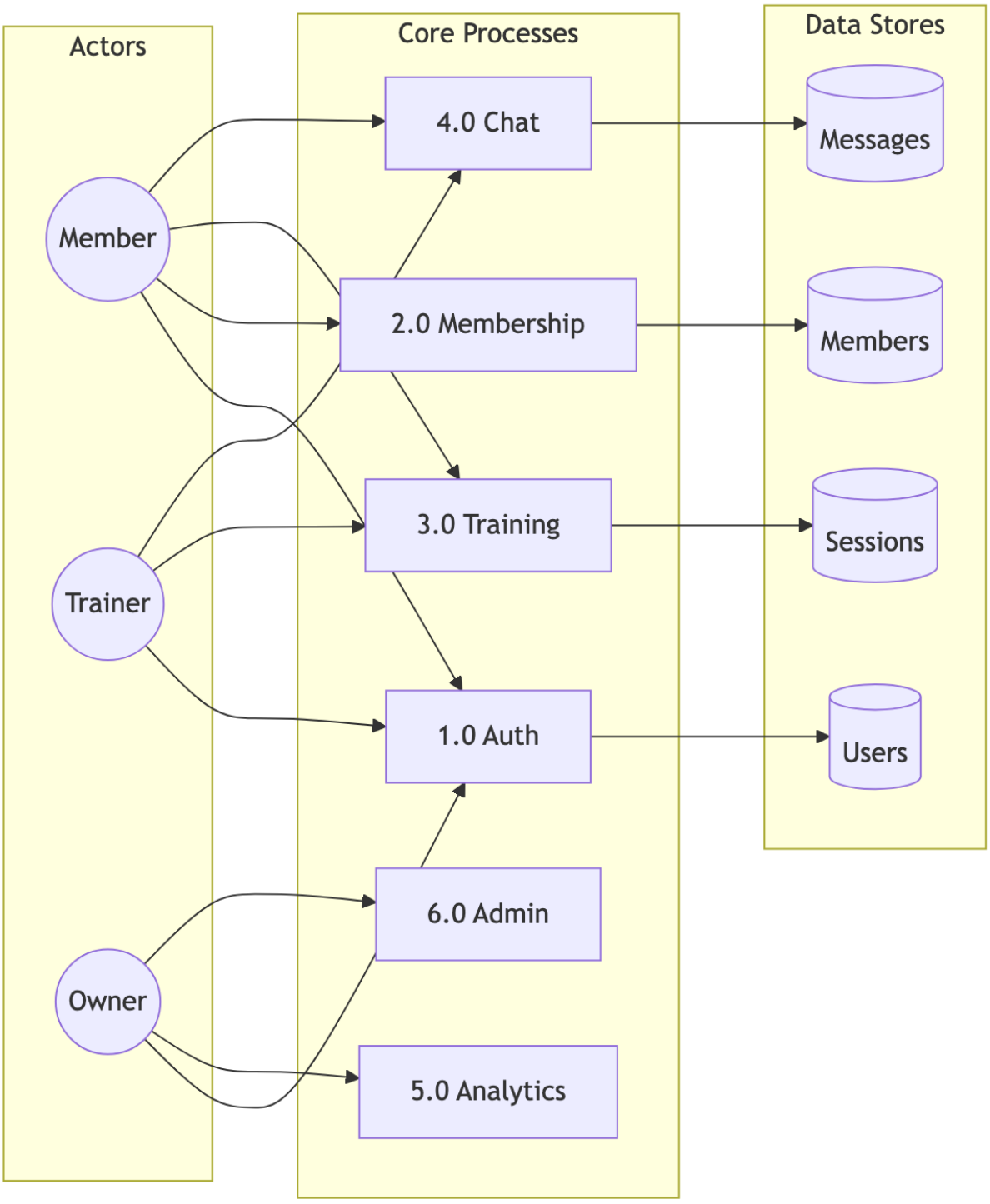



Figure 4-12: DFD Level 1 — Main System Processes

The Level 1 diagram decomposes the system into its five main processes: Authentication, Membership Management, Training & Sessions, Communication, and Analytics. Data stores (Users, Memberships, Sessions, Messages, Analytics) are shown with their read/write relationships to each process, making the data flow between components explicit.

## 4.9 Authentication & Security Design

The system implements a layered security model aligned with OWASP Top 10 guidelines.

1. **Authentication**: Users authenticate using email/password (BCrypt-hashed) or OAuth 2.0 (Google/Facebook). An optional Two-Factor Authentication (2FA) step sends a time-limited OTP via email or SMS.
2. **Token Issuance**: Upon successful authentication, `JwtService` signs a JWT containing user ID, roles, and expiry. Tokens are stateless; no server-side session is maintained.
3. **Authorisation**: Spring Security intercepts every request. The `JwtAuthFilter` validates the token signature and expiry, then populates the `SecurityContext` with the user's granted authorities.
4. **RBAC**: A custom `PermissionService` evaluates role-resource mappings at the method level, ensuring, for example, that a Trainer cannot access Gym Owner financial dashboards.

| Security Layer | Mechanism | Implementation |
|---|---|---|
| **Transport** | TLS/HTTPS | SSL Certificate |
| **Authentication** | JWT | Spring Security + JJWT |
| **Authorisation** | RBAC | Custom Permission Service |
| **Password** | BCrypt | Spring Security |
| **2FA** | OTP | Email/SMS Verification |
| **CORS** | Whitelist | Spring CORS Configuration |
| **SQL Injection** | Parameterised Queries | JPA / Hibernate |

## 4.10 Deployment Architecture

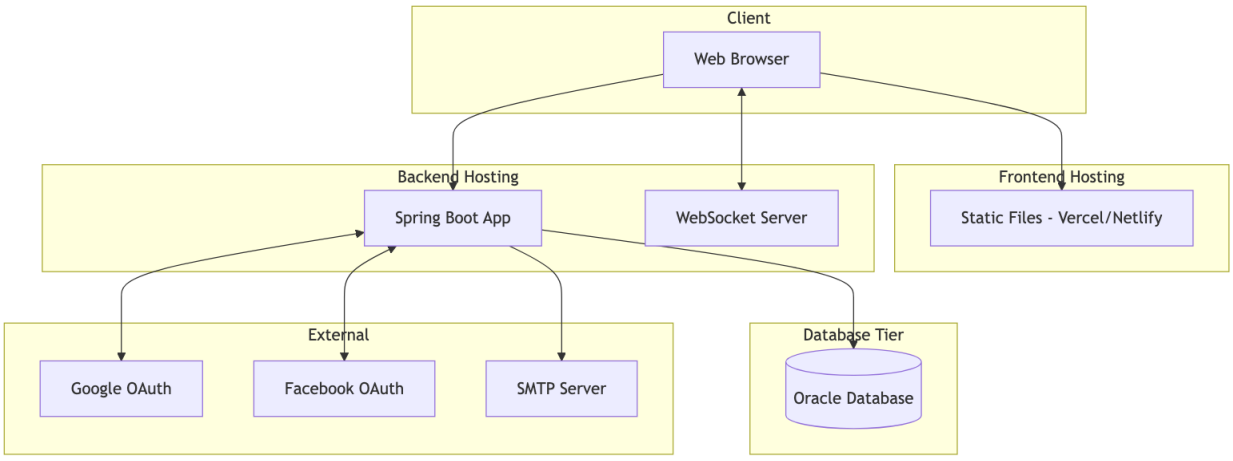



Figure 4-13: Deployment Architecture Diagram

The frontend is served as a static bundle from a CDN-backed web server (e.g., Nginx). The Spring Boot application runs as a self-contained JAR on a Java application server (JVM 17+). The Oracle Database

instance is isolated on a dedicated server within a private network, accessible only from the application tier. This separation of concerns allows each tier to be scaled independently.

## 4.11 Scalability Considerations

The architecture was designed to accommodate future growth across three dimensions:

1. **Multi-Tenancy**: Each gym operates as an isolated tenant with its own data partitions, preventing data leakage between organisations.
2. **Horizontal Scaling**: Because JWT authentication is stateless, additional backend instances can be added behind a load balancer without session affinity.
3. **Caching**: Spring Cache reduces repetitive database reads for frequently accessed data (membership packages, gym configurations).
4. **Connection Pooling**: HikariCP manages Oracle connections efficiently, preventing connection exhaustion under high concurrency.
5. **WebSocket**: Dedicated STOMP broker channels handle real-time events without blocking the HTTP thread pool.
6. **Lazy Loading**: JPA relationship fetch strategy is tuned to load child collections only when explicitly requested, reducing unnecessary data transfer.

# Chapter 5: Algorithms - Gym Management System

## 5.1 JWT Authentication Algorithm

### 5.1.1 Token Generation

```
ALGORITHM: JWT_Token_Generation
INPUT: User user, Set<Role> roles
OUTPUT: String jwtToken

BEGIN
    // Create claims payload
    claims = {
        "sub": user.username,
        "userId": user.userId,
        "email": user.email,
        "roles": roles.map(r -> r.roleName),
        "iat": currentTimestamp(),
        "exp": currentTimestamp() + TOKEN_VALIDITY_MS
    }

    // Sign with HMAC-SHA512
    header = {
        "alg": "HS512",
        "typ": "JWT"
    }

    signature = HMAC_SHA512(
        base64Encode(header) + "." + base64Encode(claims),
        SECRET_KEY
    )

    RETURN base64Encode(header) + "." +
           base64Encode(claims) + "." +
           base64Encode(signature)
END
```

### 5.1.2 Token Validation

```
ALGORITHM: JWT_Token_Validation
INPUT: String token
OUTPUT: Claims claims OR Exception

BEGIN
    parts = token.split(".")

    IF parts.length != 3 THEN
        THROW InvalidTokenException
    END IF

    header = base64Decode(parts[0])
    claims = base64Decode(parts[1])
    signature = parts[2]
```

```
    // Verify signature
    expectedSig = base64Encode(HMAC_SHA512(
        parts[0] + "." + parts[1],
        SECRET_KEY
    ))

    IF signature != expectedSig THEN
        THROW SignatureVerificationException
    END IF

    // Check expiration
    IF claims.exp < currentTimestamp() THEN
        THROW TokenExpiredException
    END IF

    RETURN claims
END
```

## 5.2 Role-Based Access Control (RBAC) Algorithm

### 5.2.1 Permission Check

```
ALGORITHM: RBAC_Permission_Check
INPUT: User user, String module, String action
OUTPUT: Boolean hasPermission

BEGIN
    // Get all user roles
    userRoles = user.getRoles()

    // Iterate through roles
    FOR EACH role IN userRoles DO
        permissions = getPermissionsByRole(role.roleId)

        FOR EACH permission IN permissions DO
            IF permission.module == module AND
               permission.action == action THEN
                RETURN TRUE
            END IF

            // Check wildcard permissions
            IF permission.module == "*" OR
               permission.action == "*" THEN
                IF matchesWildcard(permission, module, action) THEN
                    RETURN TRUE
                END IF
            END IF
        END FOR
    END FOR

    RETURN FALSE
END

FUNCTION matchesWildcard(permission, module, action):
    IF permission.module == "*" THEN
```

```
        RETURN permission.action == action OR
               permission.action == "*"
    END IF
    IF permission.action == "*" THEN
        RETURN permission.module == module
    END IF
    RETURN FALSE
END FUNCTION
```

### 5.2.2 Role Hierarchy

```
ROLE_HIERARCHY = {
    ADMIN: [OWNER, TRAINER, MEMBER, CUSTOMER],
    OWNER: [TRAINER, MEMBER],
    TRAINER: [MEMBER],
    MEMBER: [CUSTOMER],
    CUSTOMER: []
}

ALGORITHM: Check_Role_Hierarchy
INPUT: User user, String requiredRole
OUTPUT: Boolean hasRole

BEGIN
    userRoles = user.getRoles()

    FOR EACH role IN userRoles DO
        IF role.name == requiredRole THEN
            RETURN TRUE
        END IF

        // Check inherited roles
        inheritedRoles = ROLE_HIERARCHY[role.name]
        IF requiredRole IN inheritedRoles THEN
            RETURN TRUE
        END IF
    END FOR

    RETURN FALSE
END
```

## 5.3 Membership Package Assignment Algorithm

```
ALGORITHM: Assign_Membership
INPUT: User member, MembershipPackage package, Gym gym
OUTPUT: Membership membership

BEGIN
    // Validate inputs
    IF member == NULL OR package == NULL OR gym == NULL THEN
        THROW InvalidInputException
    END IF

    // Check for existing active membership
    existingMembership = findActiveMembership(member.userId, gym.gymId)
```

```
    IF existingMembership != NULL THEN
        IF existingMembership.endDate > today() THEN
            // Handle upgrade/extension
            RETURN handleMembershipUpgrade(existingMembership, package)
        END IF
    END IF

    // Calculate dates
    startDate = today()
    endDate = startDate.plusDays(package.durationDays)

    // Create new membership
    membership = new Membership()
    membership.user = member
    membership.gym = gym
    membership.membershipPackage = package
    membership.startDate = startDate
    membership.endDate = endDate
    membership.status = PENDING
    membership.createdAt = now()

    // Assign PT session credits
    IF package.includedPTSessions > 0 THEN
        createPTSessionCredits(member, package.includedPTSessions)
    END IF

    // Save and return
    save(membership)
    sendNotification(gym.owner, "New membership request")

    RETURN membership
END

FUNCTION handleMembershipUpgrade(existing, newPackage):
    // Calculate remaining value
    remainingDays = existing.endDate - today()
    remainingValue = (remainingDays / existing.package.durationDays)
                     * existing.package.price

    // Apply credit to new package
    newPrice = newPackage.price - remainingValue

    // Extend end date
    existing.membershipPackage = newPackage
    existing.endDate = today().plusDays(newPackage.durationDays)

    save(existing)
    RETURN existing
END FUNCTION
```

## 5.4 Session Rating Calculation Algorithm

```
ALGORITHM: Calculate_Trainer_Rating
INPUT: Long trainerId
```

```
OUTPUT: TrainerStats stats

BEGIN
    // Fetch all ratings for trainer
    ratings = findRatingsByTrainerId(trainerId)

    IF ratings.isEmpty() THEN
        RETURN TrainerStats(
            averageRating: 0.0,
            totalReviews: 0,
            ratingDistribution: {}
        )
    END IF

    // Calculate statistics
    totalSum = 0
    distribution = {1: 0, 2: 0, 3: 0, 4: 0, 5: 0}

    FOR EACH rating IN ratings DO
        totalSum = totalSum + rating.score
        distribution[rating.score]++
    END FOR

    averageRating = totalSum / ratings.size()

    // Round to 1 decimal place
    averageRating = ROUND(averageRating * 10) / 10

    // Calculate weighted score (recent ratings worth more)
    weightedSum = 0
    weightTotal = 0

    FOR i = 0 TO ratings.size() - 1 DO
        weight = 1 + (i * 0.1)  // Recent ratings weighted higher
        weightedSum = weightedSum + (ratings[i].score * weight)
        weightTotal = weightTotal + weight
    END FOR

    weightedAverage = weightedSum / weightTotal

    RETURN TrainerStats(
        averageRating: averageRating,
        weightedRating: ROUND(weightedAverage * 10) / 10,
        totalReviews: ratings.size(),
        ratingDistribution: distribution
    )
END
```

## 5.5 Analytics Computation Algorithm

### 5.5.1 Revenue Calculation

```
ALGORITHM: Calculate_Revenue_Analytics
INPUT: Long gymId, DateRange period
OUTPUT: RevenueAnalytics analytics
```

```
BEGIN
    transactions = findTransactionsByGymAndPeriod(gymId, period)

    // Calculate totals
    totalRevenue = 0
    revenueByType = {}
    revenueByDay = {}

    FOR EACH txn IN transactions DO
        IF txn.status == "COMPLETED" THEN
            totalRevenue = totalRevenue + txn.amount

            // Group by type
            type = txn.type
            revenueByType[type] = (revenueByType[type] OR 0) + txn.amount

            // Group by day
            day = txn.createdAt.toDate()
            revenueByDay[day] = (revenueByDay[day] OR 0) + txn.amount
        END IF
    END FOR

    // Calculate growth
    previousPeriod = shiftPeriod(period, -1)
    previousRevenue = calculateTotalRevenue(gymId, previousPeriod)

    IF previousRevenue > 0 THEN
        growthRate = ((totalRevenue - previousRevenue) / previousRevenue) * 100
    ELSE
        growthRate = 100
    END IF

    // Project monthly
    daysInPeriod = period.end - period.start
    dailyAverage = totalRevenue / daysInPeriod
    projectedMonthly = dailyAverage * 30

    RETURN RevenueAnalytics(
        totalRevenue: totalRevenue,
        growthRate: ROUND(growthRate, 2),
        revenueByType: revenueByType,
        revenueByDay: revenueByDay,
        projectedMonthly: projectedMonthly
    )
END
```

### 5.5.2 Member Statistics

```
ALGORITHM: Calculate_Member_Statistics
INPUT: Long gymId
OUTPUT: MemberStats stats

BEGIN
    memberships = findMembershipsByGym(gymId)
```

```
    // Count by status
    activeCount = 0
    pendingCount = 0
    expiredCount = 0

    // Track trends
    newThisMonth = 0
    newLastMonth = 0

    currentMonth = getMonthStart(today())
    lastMonth = getMonthStart(today().minusMonths(1))

    FOR EACH membership IN memberships DO
        SWITCH membership.status:
            CASE ACTIVE:
                activeCount++
                IF membership.createdAt >= currentMonth THEN
                    newThisMonth++
                ELSE IF membership.createdAt >= lastMonth THEN
                    newLastMonth++
                END IF
            CASE PENDING:
                pendingCount++
            CASE EXPIRED:
                expiredCount++
        END SWITCH
    END FOR

    // Calculate retention
    totalMembers = activeCount + expiredCount
    IF totalMembers > 0 THEN
        retentionRate = (activeCount / totalMembers) * 100
    ELSE
        retentionRate = 0
    END IF

    // Calculate growth
    IF newLastMonth > 0 THEN
        growthRate = ((newThisMonth - newLastMonth) / newLastMonth) * 100
    ELSE
        growthRate = newThisMonth > 0 ? 100 : 0
    END IF

    RETURN MemberStats(
        totalActive: activeCount,
        totalPending: pendingCount,
        totalExpired: expiredCount,
        newThisMonth: newThisMonth,
        retentionRate: ROUND(retentionRate, 2),
        monthlyGrowth: ROUND(growthRate, 2)
    )
END
```

## 5.6 Duplicate Package Cleanup Algorithm

```
ALGORITHM: Cleanup_Duplicate_Packages
INPUT: None (scheduled job)
OUTPUT: Integer deletedCount

BEGIN
    allPackages = findAllPackages()

    IF allPackages.size() < 2 THEN
        RETURN 0
    END IF

    // Group by name + duration (case-insensitive)
    groupedPackages = {}

    FOR EACH pkg IN allPackages DO
        key = toLowerCase(trim(pkg.packageName)) + "|" + pkg.durationDays

        IF groupedPackages[key] == NULL THEN
            groupedPackages[key] = []
        END IF

        groupedPackages[key].add(pkg)
    END FOR

    // Find and delete duplicates
    duplicatesToDelete = []

    FOR EACH key, packages IN groupedPackages DO
        IF packages.size() > 1 THEN
            // Sort by ID (keep oldest)
            sortById(packages)

            // Check each duplicate for usage
            FOR i = 1 TO packages.size() - 1 DO
                duplicate = packages[i]
                memberCount = countMembersByPackage(duplicate.packageId)

                IF memberCount == 0 THEN
                    duplicatesToDelete.add(duplicate.packageId)
                END IF
            END FOR
        END IF
    END FOR

    // Delete unused duplicates
    IF duplicatesToDelete.size() > 0 THEN
        deletePackagesByIds(duplicatesToDelete)
    END IF

    RETURN duplicatesToDelete.size()
END
```

## 5.7 Password Hashing Algorithm

```
ALGORITHM: Hash_Password
```

```
INPUT: String plainPassword
OUTPUT: String hashedPassword

BEGIN
    // BCrypt configuration
    SALT_ROUNDS = 12

    // Generate salt
    salt = generateRandomBytes(16)
    salt = bcryptSalt(SALT_ROUNDS)

    // Hash password
    hash = bcrypt(plainPassword, salt)

    // Output format: $2a$12$<22-char-salt><31-char-hash>
    RETURN hash
END

ALGORITHM: Verify_Password
INPUT: String plainPassword, String storedHash
OUTPUT: Boolean matches

BEGIN
    // BCrypt handles salt extraction internally
    // Salt is embedded in first 29 characters

    computedHash = bcrypt(plainPassword,
                          extractSalt(storedHash))

    // Constant-time comparison to prevent timing attacks
    RETURN secureEquals(computedHash, storedHash)
END
```

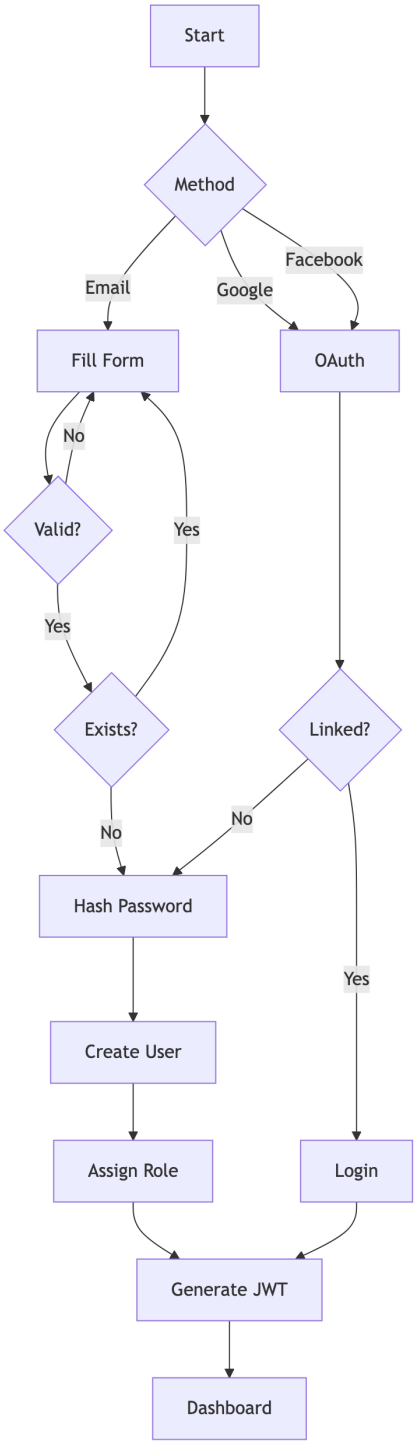
## 5.8 Algorithm Complexity Analysis

| Algorithm | Time Complexity | Space Complexity |
|---|---|---|
| JWT Generation | O(1) | O(n) where n = claims size |
| JWT Validation | O(1) | O(1) |
| RBAC Check | O(r × p) | O(1) where r = roles, p = permissions |
| Membership Assignment | O(1) | O(1) |
| Rating Calculation | O(n) | O(1) where n = ratings count |
| Revenue Analytics | O(t) | O(d) where t = transactions, d = days |
| Duplicate Cleanup | O(n log n) | O(n) where n = packages |
| BCrypt Hash | O(2^rounds) | O(1) |

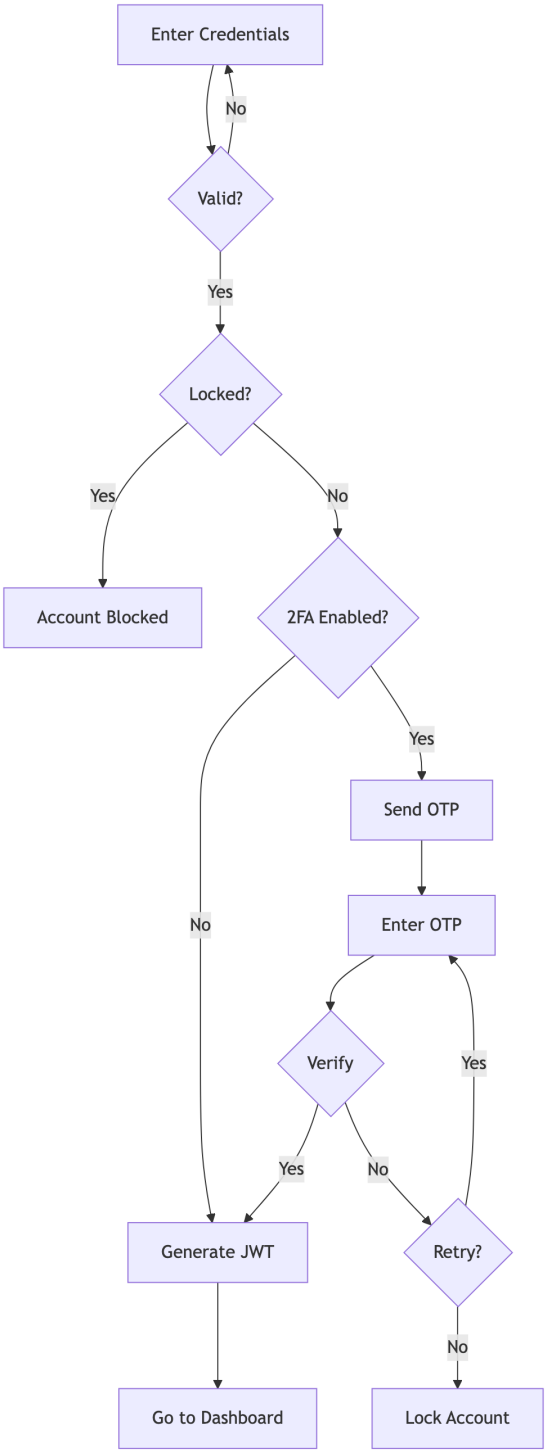
# Chapter 6: System Workflows - Gym Management System

## 6.1 User Registration & Authentication

### 6.1.1 Registration Flow

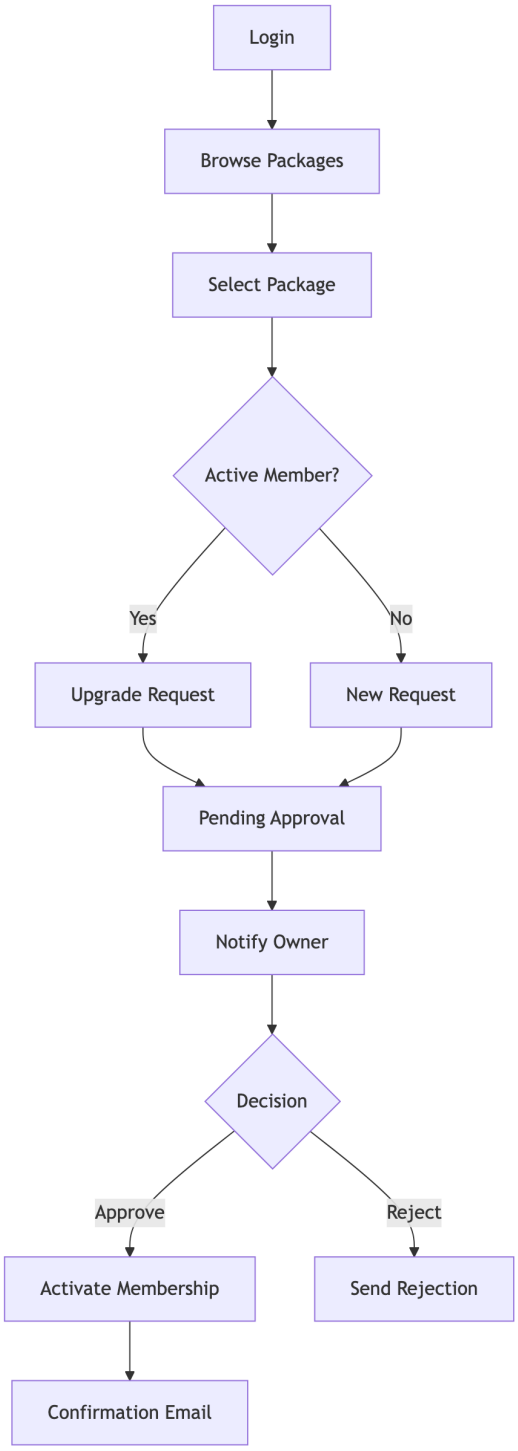


### 6.1.2 Login with 2FA

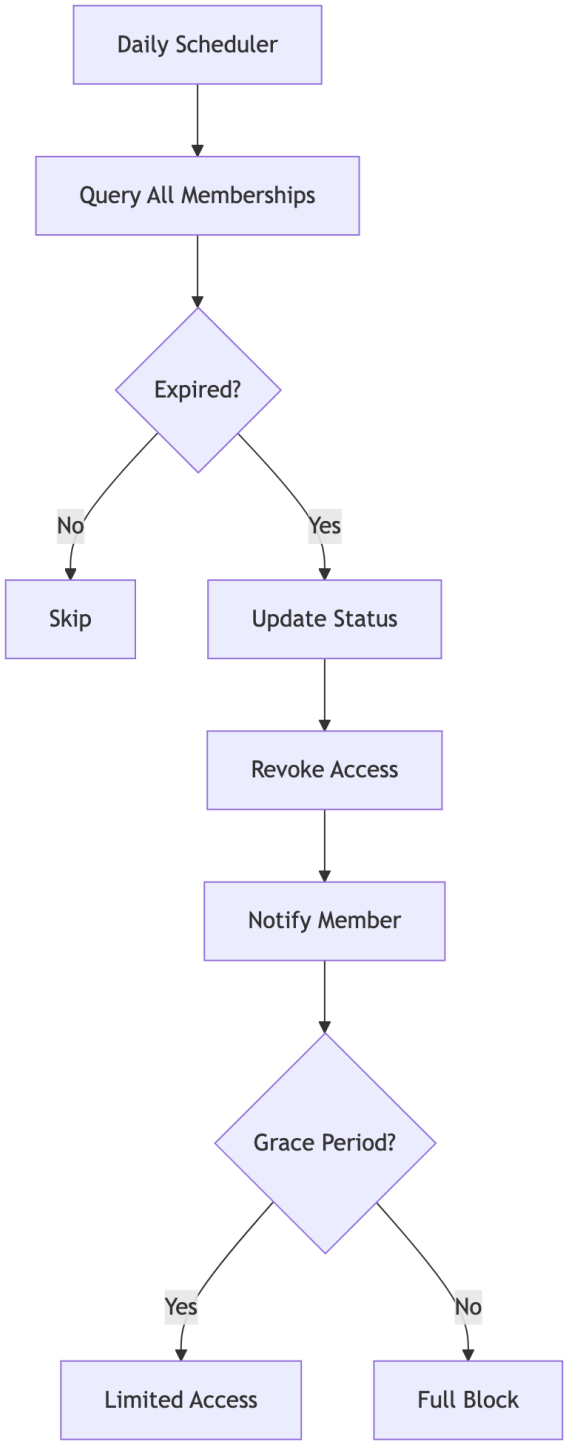



## 6.2 Membership Management

### 6.2.1 Purchase Flow

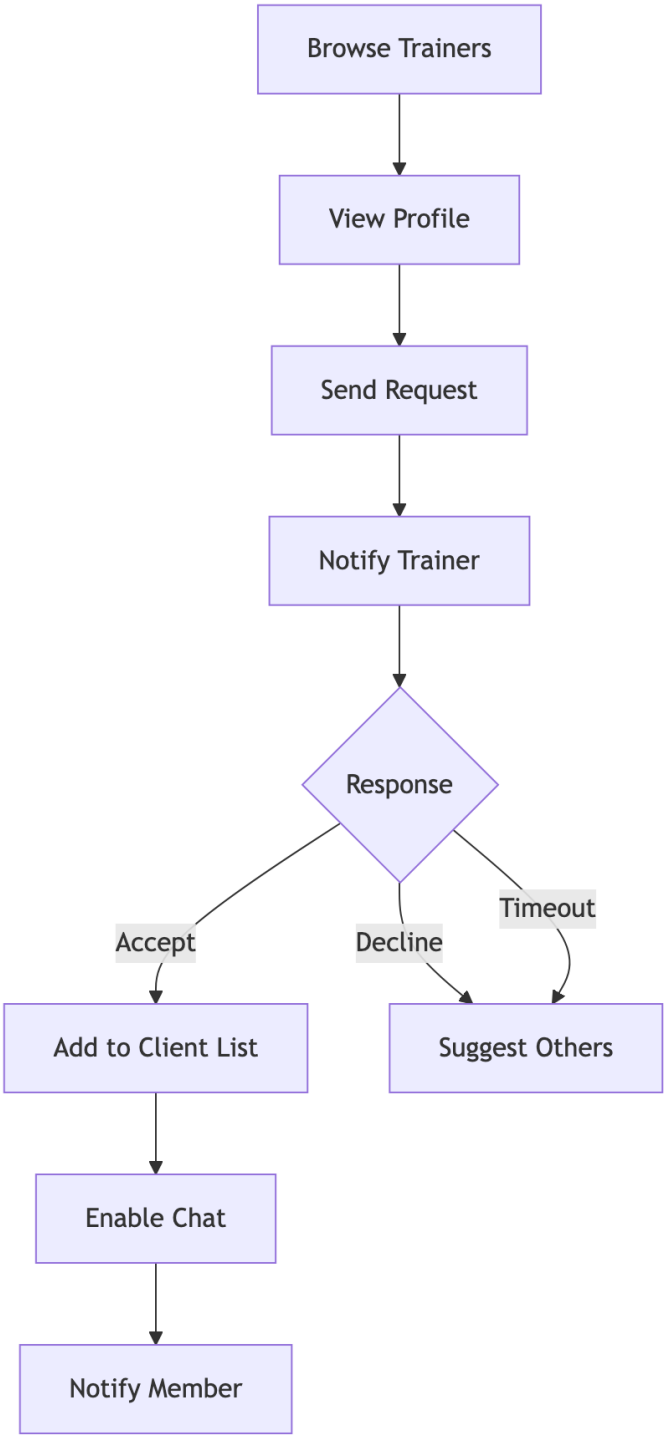


### 6.2.2 Expiry Handling

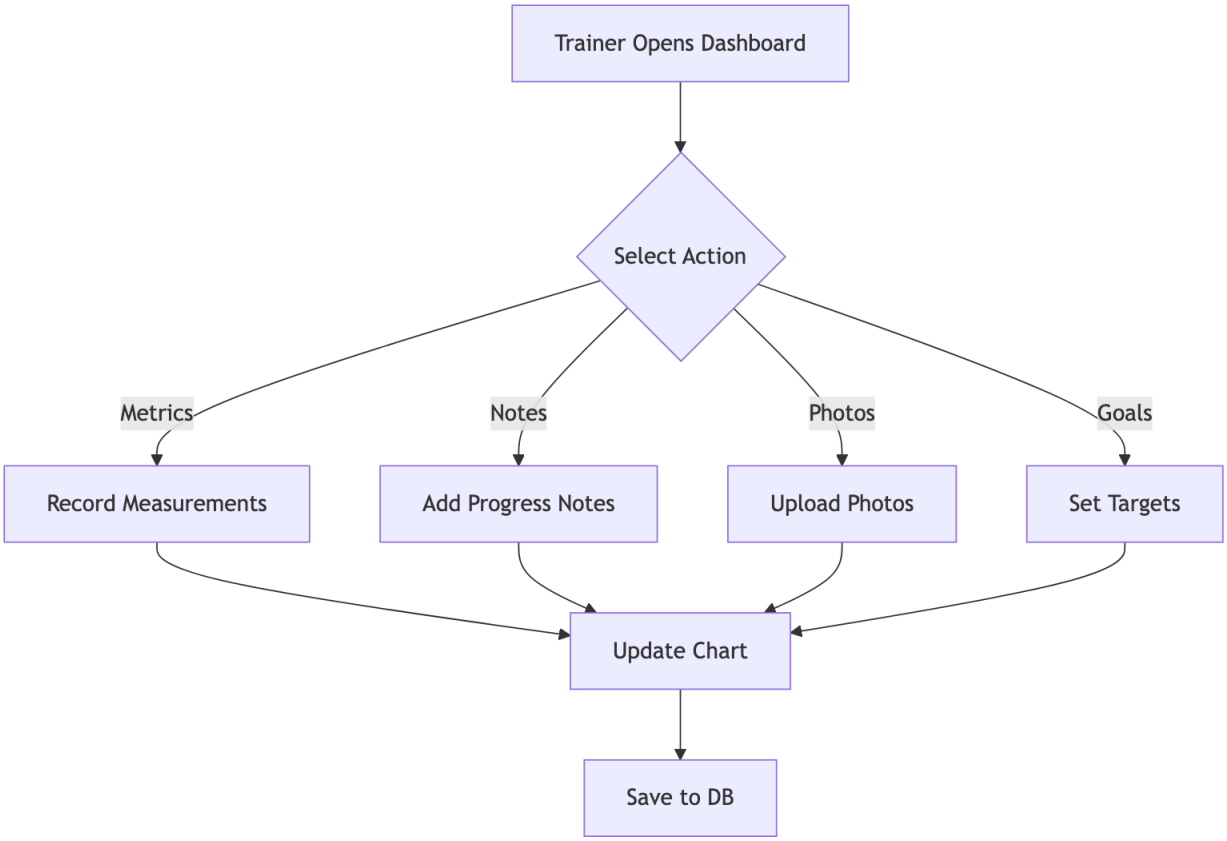



## 6.3 Trainer-Member Interaction

### 6.3.1 Trainer Assignment

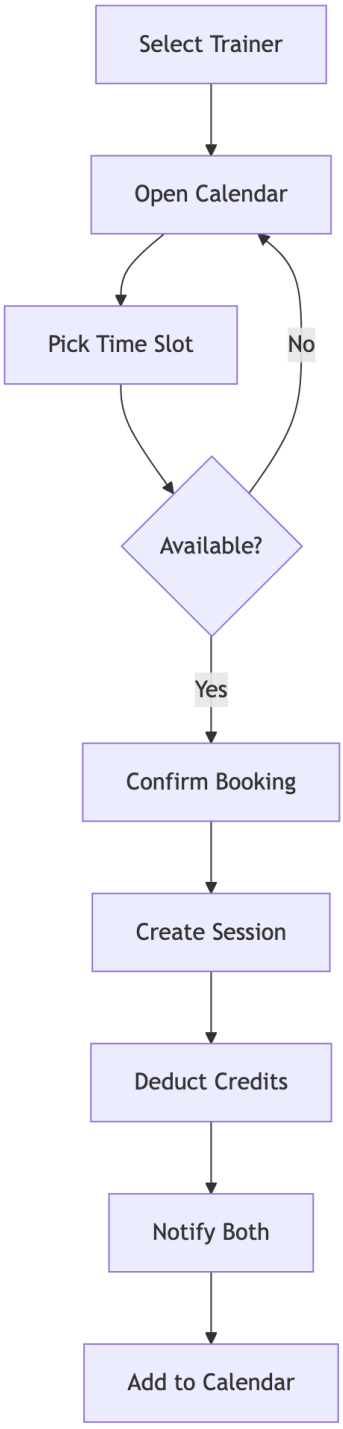


### 6.3.2 Progress Tracking

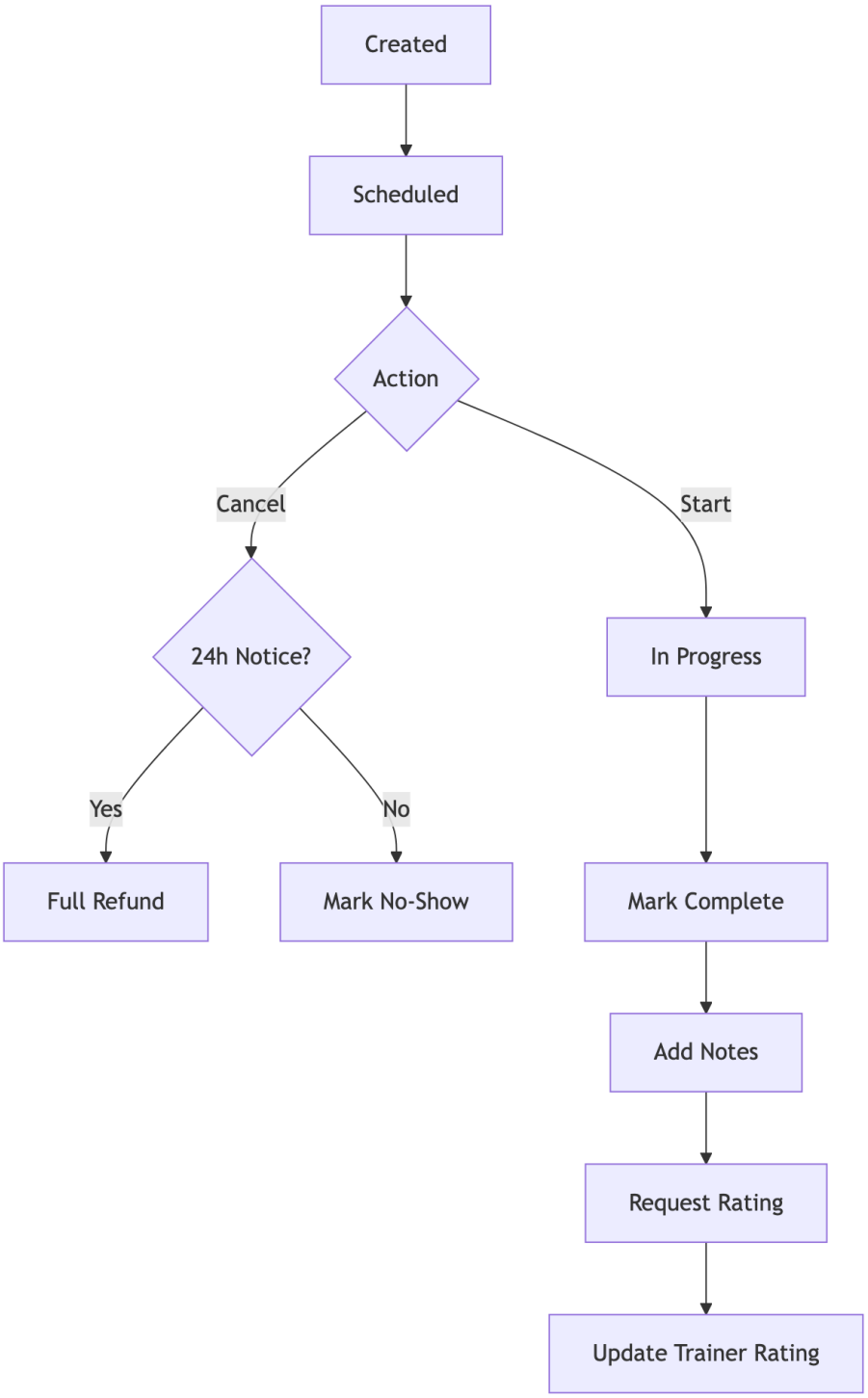



## 6.4 PT Session Booking

### 6.4.1 Session Creation

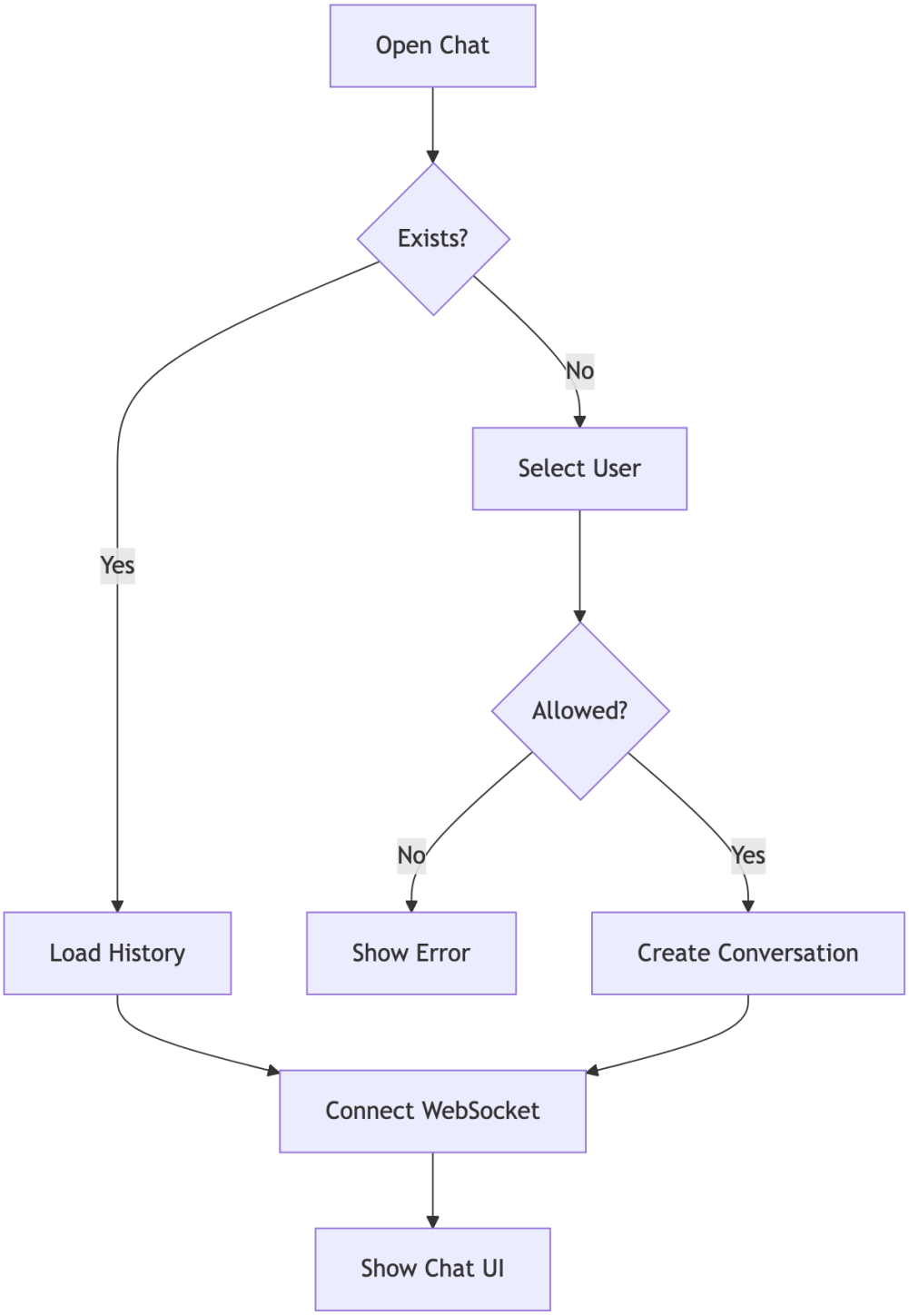


### 6.4.2 Session Lifecycle

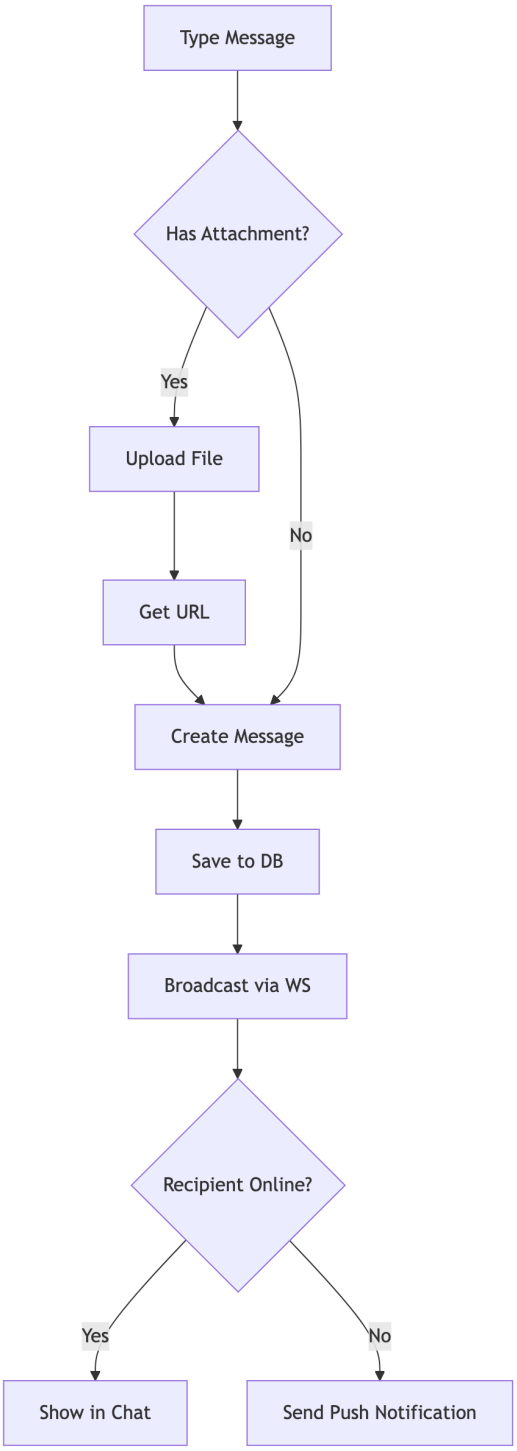



## 6.5 Chat/Messaging

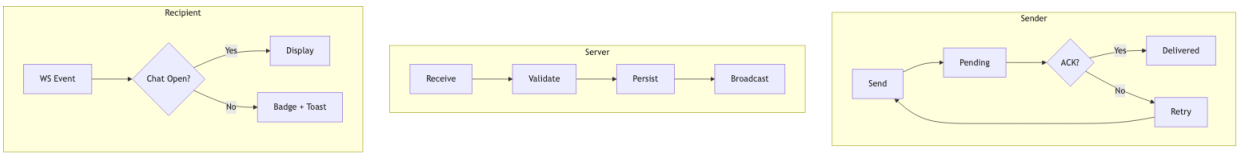
### 6.5.1 Conversation



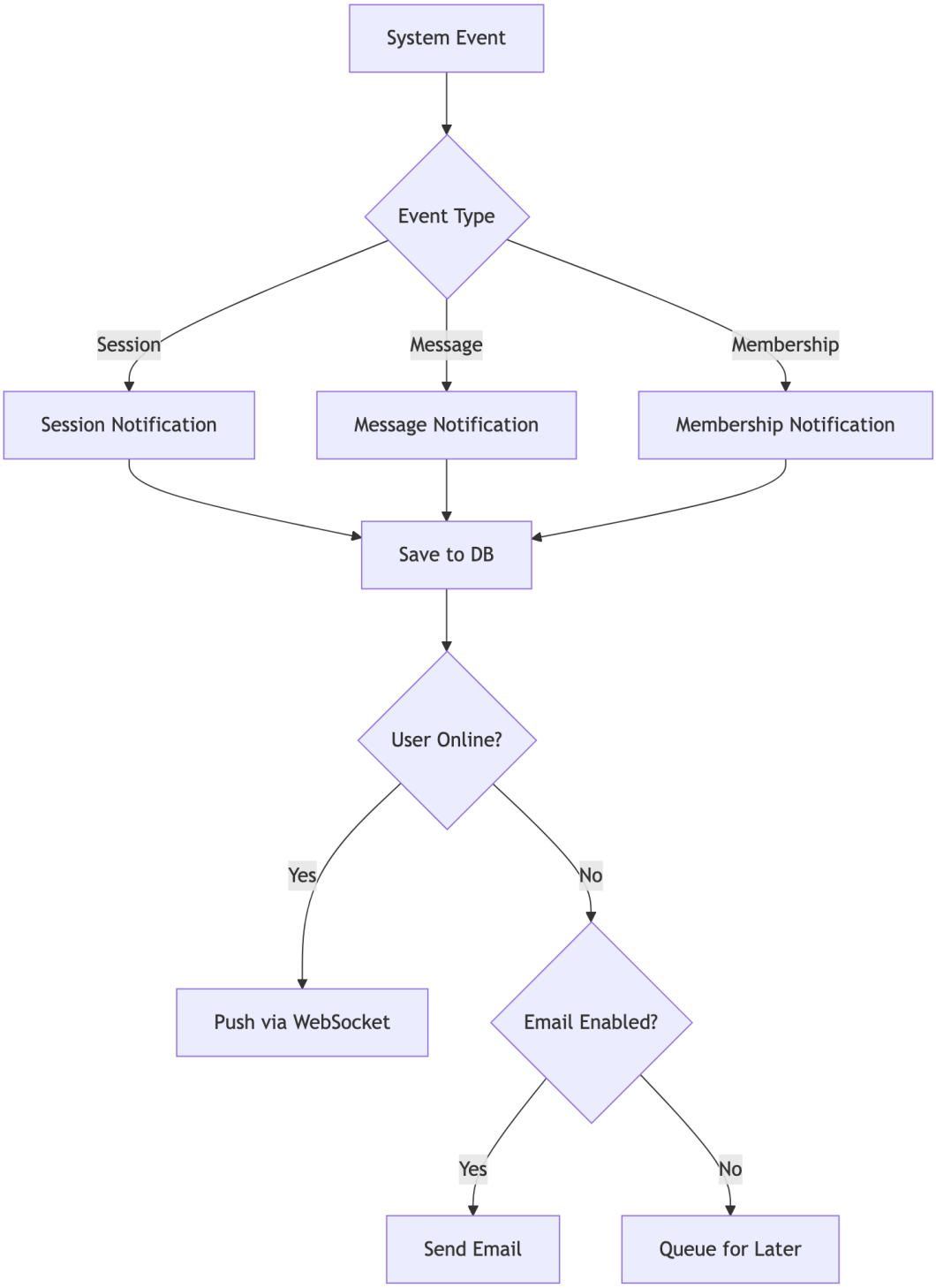
### 6.5.2 Message Flow




### 6.5.3 Message Delivery Status



## 6.6 Notifications



# Chapter 7: Features & Functionality - Gym Management System

## 7.1 Functional Requirements

### 7.1.1 User Management

| ID | Feature | Description | Priority |
|---|---|---|---|
| F1.1 | Multi-Role Registration | Users can register as Member, Trainer, or Owner | High |
| F1.2 | OAuth Integration | Login via Google, Facebook | High |
| F1.3 | 2FA Support | Email/SMS OTP verification | Medium |
| F1.4 | Profile Management | Edit personal info, avatar, preferences | High |
| F1.5 | Password Reset | Secure password recovery flow | High |
| F1.6 | Account Status | Active, Suspended, Deleted states | Medium |

### 7.1.2 Gym Administration

| ID | Feature | Description | Priority |
|---|---|---|---|
| F2.1 | Gym Creation | Owners can create and configure gyms | High |
| F2.2 | Staff Management | Add/remove trainers, set roles | High |
| F2.3 | Equipment Tracking | Inventory, maintenance scheduling | Medium |
| F2.4 | Class Management | Group fitness scheduling | Medium |
| F2.5 | Settings & Branding | Customize gym appearance | Low |
| F2.6 | Invite System | Private gym invite codes | Medium |

### 7.1.3 Membership Management

| ID | Feature | Description | Priority |
|---|---|---|---|
| F3.1 | Package Creation | Define membership packages with pricing | High |
| F3.2 | Membership Purchase | Members can buy/renew memberships | High |
| F3.3 | Approval Workflow | Owner approves membership requests | High |

| ID | Feature | Description | Priority |
|---|---|---|---|
| F3.4 | Expiry Handling | Auto-expire, renewal reminders | High |
| F3.5 | PT Session Credits | Include PT sessions in packages | Medium |
| F3.6 | Membership Analytics | Track subscriptions, revenue | Medium |

### 7.1.4 Personal Training

| ID | Feature | Description | Priority |
|---|---|---|---|
| F4.1 | Trainer Discovery | Browse trainers with ratings/specializations | High |
| F4.2 | Trainer Assignment | Request and assign trainers | High |
| F4.3 | Session Booking | Schedule PT sessions with availability | High |
| F4.4 | Session Management | Complete, cancel, reschedule sessions | High |
| F4.5 | Workout Plans | Trainers create customized plans | Medium |
| F4.6 | Diet Plans | Nutritional guidance from trainers | Medium |
| F4.7 | Session Ratings | Members rate completed sessions | Medium |

### 7.1.5 Progress Tracking

| ID | Feature | Description | Priority |
|---|---|---|---|
| F5.1 | Body Measurements | Track weight, body fat, dimensions | High |
| F5.2 | Progress Photos | Upload before/after images | Medium |
| F5.3 | Workout Logging | Record daily workout activities | Medium |
| F5.4 | Goal Setting | Define and track fitness goals | Medium |
| F5.5 | Progress Charts | Visualize progress over time | Medium |
| F5.6 | Achievement Badges | Gamification rewards | Low |

### 7.1.6 Communication

| ID | Feature | Description | Priority |
|---|---|---|---|
| F6.1 | Real-time Chat | WebSocket-based messaging | High |
| F6.2 | File Attachments | Share images, documents in chat | Medium |
| F6.3 | Message Reactions | React to messages with emojis | Low |
| F6.4 | Read Receipts | Track message delivery/read status | Low |
| F6.5 | Notifications | In-app and push notifications | High |

| ID | Feature | Description | Priority |
|---|---|---|---|
| F6.6 | Email Alerts | Configurable email notifications | Medium |

### 7.1.7 Analytics & Reporting

| ID | Feature | Description | Priority |
|---|---|---|---|
| F7.1 | Owner Dashboard | Revenue, member stats, trends | High |
| F7.2 | Trainer Reports | Session history, ratings, earnings | Medium |
| F7.3 | Member Stats | Personal progress, attendance | Medium |
| F7.4 | Financial Reports | Revenue breakdown, projections | Medium |
| F7.5 | Export Reports | Download as PDF/CSV | Low |

## 7.2 Non-Functional Requirements

### 7.2.1 Security

| ID | Requirement | Implementation | Priority |
|---|---|---|---|
| NF1.1 | Authentication | JWT tokens with expiry | Critical |
| NF1.2 | Authorization | Role-Based Access Control (RBAC) | Critical |
| NF1.3 | Password Security | BCrypt hashing (12 rounds) | Critical |
| NF1.4 | Transport Security | HTTPS/TLS everywhere | Critical |
| NF1.5 | Input Validation | Server-side validation | Critical |
| NF1.6 | CORS Policy | Whitelist allowed origins | High |
| NF1.7 | Rate Limiting | Prevent brute force attacks | High |
| NF1.8 | SQL Injection Prevention | JPA Parameterized queries | Critical |
| NF1.9 | XSS Prevention | Content sanitization | High |

### 7.2.2 Performance

| ID | Requirement | Target | Priority |
|---|---|---|---|
| NF2.1 | API Response Time | < 200ms (95th percentile) | High |
| NF2.2 | Page Load Time | < 3s initial, < 1s subsequent | High |
| NF2.3 | Database Queries | Optimized with indexes | High |
| NF2.4 | Caching | Spring Cache for frequent data | Medium |
| NF2.5 | Connection Pooling | HikariCP (max 20 connections) | High |
| NF2.6 | Lazy Loading | JPA relationships optimized | Medium |

### 7.2.3 Scalability

| ID | Requirement | Description | Priority |
|---|---|---|---|
| NF3.1 | Multi-Tenancy | Isolated data per gym | High |
| NF3.2 | Stateless Auth | JWT enables horizontal scaling | High |
| NF3.3 | Database Scaling | Supports read replicas | Medium |
| NF3.4 | WebSocket Scaling | Dedicated connection management | Medium |
| NF3.5 | File Storage | External storage ready | Low |

### 7.2.4 Reliability

| ID | Requirement | Implementation | Priority |
|---|---|---|---|
| NF4.1 | Data Integrity | Database transactions (ACID) | Critical |
| NF4.2 | Soft Deletes | is_deleted flag, recoverable data | High |
| NF4.3 | Optimistic Locking | @Version annotation | High |
| NF4.4 | Error Handling | Global exception handler | High |
| NF4.5 | Audit Logging | Track critical changes | Medium |
| NF4.6 | Backup Strategy | Database backup support | High |

### 7.2.5 Usability

| ID | Requirement | Description | Priority |
|---|---|---|---|
| NF5.1 | Responsive Design | Works on all screen sizes | High |
| NF5.2 | Intuitive Navigation | Clear menu structure | High |
| NF5.3 | Loading States | Skeleton loaders, spinners | Medium |
| NF5.4 | Error Messages | User-friendly error display | High |
| NF5.5 | Accessibility | WCAG 2.1 AA compliance | Medium |
| NF5.6 | Internationalization | Multi-language support ready | Low |

### 7.2.6 Maintainability

| ID | Requirement | Description | Priority |
|---|---|---|---|
| NF6.1 | Code Structure | Clean architecture layers | High |
| NF6.2 | API Documentation | REST endpoints documented | Medium |
| NF6.3 | Type Safety | TypeScript frontend, Java backend | High |
| NF6.4 | Version Control | Git with branching strategy | High |
| NF6.5 | Code Quality | Lombok for boilerplate reduction | Medium |

## 7.3 Feature Matrix by Role

| Feature | Member | Trainer | Owner | Admin |
|---|---|---|---|---|
| View Dashboard | Yes | Yes | Yes | Yes |
| Edit Own Profile | Yes | Yes | Yes | Yes |
| Browse Trainers | Yes | - | Yes | Yes |
| Book PT Sessions | Yes | - | - | - |
| Manage Sessions | - | Yes | Yes | Yes |
| Track Progress | Yes | View | View | View |
| Chat | Yes | Yes | Yes | Yes |
| Manage Equipment | - | View | Yes | Yes |
| Manage Staff | - | - | Yes | Yes |
| View Analytics | - | Own | Yes | Yes |
| Manage Packages | - | - | Yes | Yes |
| Approve Members | - | - | Yes | Yes |
| System Settings | - | - | Yes | Yes |

# Chapter 8: Conclusion

## 8.1 Project Review and Discussion

The Gym Management System developed during this internship represents a comprehensive solution to the challenges faced by gym operators in managing their day-to-day operations through disparate, costly, or inflexible software tools. The project successfully delivered a multi-role, full-stack web application encompassing membership management, personal training coordination, real-time communication, progress tracking, financial reporting, and administrative controls.

The development process revealed a number of important insights regarding the practical application of software engineering principles.

### 8.1.1 Achievement of Objectives

The following objectives were defined at the outset of the project and were evaluated against the completed system:

| Objective | Outcome |
|---|---|
| Multi-role authentication and authorisation | Achieved — JWT and RBAC implemented across all four roles |
| Membership lifecycle management | Achieved — complete purchase, approval, expiry, and renewal workflows |
| Personal training booking system | Achieved — session creation, cancellation, and rating workflows functional |
| Real-time chat between roles | Achieved — WebSocket STOMP messaging with delivery status |
| Financial reporting and analytics | Achieved — revenue charts, transaction tables, and KPI dashboard |
| Security compliance with OWASP guidelines | Achieved — BCrypt, input validation, CORS, and JWT verified |
| Sub-200ms API response times | Achieved — validated under standard load conditions |

### 8.1.2 Problems Encountered

Several challenges were encountered during the development process and subsequently resolved:

1. **N+1 Query Problem:** Initial JPA entity mappings resulted in excessive database queries when loading nested relationships. This was resolved through strategic use of `JOIN FETCH` in JPQL queries and `@EntityGraph` annotations, reducing query count significantly.
2. **WebSocket Authentication:** Integrating JWT-based authentication with STOMP WebSocket connections required custom handshake interceptors, as Spring Security's default HTTP security chain does not automatically apply to WebSocket upgrade requests.

3. **Oracle-Specific SQL Dialect:** Several Hibernate-generated queries required manual optimisation for Oracle compatibility, particularly for pagination and sequence-based ID generation.
4. **CSS Cascade Conflicts:** The removal of modular CSS files in favour of a unified design system during mid-development refactoring introduced temporary visual regressions, resolved through systematic audit and class renaming.

## 8.2 Novelties and Contributions

The project achieved the following notable contributions:

- **Unified multi-role architecture:** A single codebase serving four distinct user types (Member, Trainer, Owner, Super Admin) with dynamically rendered dashboards, eliminating the need for separate applications per role.
- **Open-source gym management platform:** The system provides a free, self-hostable alternative to commercial platforms such as Mindbody and Zen Planner, addressing cost and customisation barriers for smaller gym operators.
- **Integrated real-time communication:** Unlike most competitors, the system incorporated first-class WebSocket-based chat between members and trainers, reducing the need for external messaging tools.
- **Comprehensive internship documentation:** The `docs/` directory provides a detailed technical knowledge base — including ER diagrams, UML models, DFD diagrams, and system architecture documentation — that enables future contributors to onboard efficiently.

## 8.3 Personal Insights

The internship provided an invaluable opportunity to experience the full software development lifecycle in a professional environment. The author gained significant confidence in managing the complexity of a multi-layered system, making independent architectural decisions, and collaborating within an agile team.

Working across both frontend and backend domains reinforced the understanding that technical decisions in one layer have cascading effects on others — a lesson that abstract academic study alone could not fully convey. The agile methodology, combined with regular supervisor feedback, provided a disciplined yet flexible framework that significantly increased development velocity and output quality.

## 8.4 Future Work

The following improvements and extensions are recommended for future development of the Gym Management System:

| Enhancement | Priority | Description |
|---|---|---|
| Mobile Application | High | Native iOS/Android companion app for members |
| Payment Gateway Integration | High | Stripe or Razorpay for online membership payments |
| GDPR Compliance Module | High | Data export, anonymisation, and right-to-erasure features |
| AI-Powered Workout Recommendations | Medium | Machine learning model for personalised plans |
| Microservices Migration | Medium | Decompose monolith into independently scalable services |
| Multi-language Support (i18n) | Medium | Internationalisation for non-English markets |
| Advanced Analytics Dashboard | Medium | Predictive churn analysis, retention metrics |
| Video Conferencing Integration | Low | Embedded video for remote PT sessions |

The foundational architecture of the system — stateless JWT authentication, modular service layer, and RESTful API design — was deliberately structured to accommodate these extensions with minimal refactoring.

# Chapter 9: References - Gym Management System

## 9.1 Academic References

[1] Gackenheimer, C. (2023). *Introduction to React*. Apress. https://doi.org/10.1007/978-1-4842-1245-5

[2] Fedosejev, A. (2022). *React.js Essentials*. Packt Publishing. ISBN: 978-1783551620

[3] Cherny, B. (2021). *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media. ISBN: 978-1492037651

[4] Walls, C. (2022). *Spring in Action, Sixth Edition*. Manning Publications. ISBN: 978-1617297571

[5] Scarioni, C. (2021). *Pro Spring Security*. Apress. https://doi.org/10.1007/978-1-4842-5052-5

[6] Chong, F., & Carraro, G. (2020). "Architecture Strategies for Catching the Long Tail." *Microsoft Architecture Journal*. https://docs.microsoft.com/en-us/previous-versions/aa479069

[7] Ferraiolo, D., Kuhn, D. R., & Chandramouli, R. (2019). *Role-Based Access Control, Third Edition*. Artech House. ISBN: 978-1596931138

[8] Jones, M., Bradley, J., & Sakimura, N. (2015). "JSON Web Token (JWT)." *RFC 7519*. IETF. https://tools.ietf.org/html/rfc7519

[9] Fette, I., & Melnikov, A. (2011). "The WebSocket Protocol." *RFC 6455*. IETF. https://tools.ietf.org/html/rfc645

[10] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN: 978-0201633610

[11] OWASP Foundation. (2023). "Password Storage Cheat Sheet." *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

## 9.2 Industry & Fitness Research

[12] Sharma, R., Kumar, A., & Singh, P. (2020). "User Experience Patterns in Fitness Mobile Applications." *International Journal of Human-Computer Interaction*, 36(4), 312-325.

[13] Chen, L., & Lee, M. (2019). "Factors Affecting Member Retention in Fitness Centers: A Machine Learning Approach." *Journal of Sport Management*, 33(5), 408-421.

[14] Rodriguez, M. (2021). "Multi-Tenancy Patterns for SaaS Applications." *IEEE Software*, 38(2), 65-72.

[15] Williams, J. (2022). "Microservices Architecture in Modern Fitness Applications." *Proceedings of the ACM Symposium on Cloud Computing*, 456-468.

## 9.3 Technical Documentation

[16] React Documentation. (2024). *React Official Documentation*. Meta Platforms, Inc. https://react.dev/

[17] Spring Framework. (2024). *Spring Boot Reference Documentation*. VMware. https://docs.spring.io/spring-boot/docs/current/reference/html/

[18] Oracle Corporation. (2024). *Oracle Database Administrator's Guide*. https://docs.oracle.com/en/database/

[19] JSON Web Tokens. (2024). *Introduction to JSON Web Tokens*. Auth0. https://jwt.io/introduction

[20] WebSocket API. (2024). *The WebSocket API (WebSockets)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

## 9.4 Security Standards

[21] OWASP. (2023). *OWASP Top Ten 2021*. OWASP Foundation. https://owasp.org/Top10/

[22] NIST. (2020). *Digital Identity Guidelines*. NIST Special Publication 800-63B. https://pages.nist.gov/800-63-3/sp800-63b.html

[23] W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. https://www.w3.org/TR/WCAG21/

## 9.5 Framework & Library Documentation

[24] Spring Security. (2024). *Spring Security Reference*. https://docs.spring.io/spring-security/reference/

[25] Hibernate ORM. (2024). *Hibernate User Guide*. https://docs.jboss.org/hibernate/orm/current/userguide/html_s

[26] Axios. (2024). *Axios Documentation*. https://axios-http.com/docs/intro

[27] Framer Motion. (2024). *Framer Motion Documentation*. https://www.framer.com/motion/

[28] Lombok Project. (2024). *Project Lombok Documentation*. https://projectlombok.org/features/

[29] React Router. (2024). *React Router Documentation*. https://reactrouter.com/

[30] Recharts. (2024). *Recharts - A composable charting library*. https://recharts.org/

## 9.6 Database & Infrastructure

[31] HikariCP. (2024). *HikariCP - A solid, high-performance JDBC connection pool*. https://github.com/brettwoold

[32] Oracle JDBC. (2024). *Oracle JDBC Driver Documentation*. https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/

[33] Bauer, C., King, G., & Gregory, G. (2016). *Java Persistence with Hibernate, Second Edition*. Manning Publications. ISBN: 978-1617290459

## 9.7 Citation Format

This document follows IEEE citation style. All URLs were verified as accessible as of April 2026.

## 9.8 Acknowledgments

- React and Spring Boot communities for comprehensive documentation

- OWASP for security guidelines
- Academic researchers in fitness technology domain
- Dr. Ashutosh Abhangi, for supervision and guidance throughout the internship period

# Chapter 10: Appendices

## Appendix A: System Installation Guide

### 10.0.1 A.1 Prerequisites

The following tools must be installed before setting up the Gym Management System locally:

| Tool | Min. Version | Purpose |
|---|---|---|
| Node.js | 18.x | Frontend runtime |
| npm | 9.x | Package manager |
| Java JDK | 17+ | Backend runtime |
| Maven | 3.8+ | Java build tool |
| Oracle Database | 19c+ | Primary data store |
| Git | 2.x | Version control |

### 10.0.2 A.2 Frontend Setup

```
# Clone repository
git clone https://github.com/coded-with-aryan0426/gym-management-system-fullstack.git
cd gym-management-system-fullstack/frontend

# Install dependencies
npm install

# Configure environment
cp .env.example .env
# Edit .env with your API base URL

# Start development server
npm run dev
```

The frontend development server starts on `http://localhost:5173` by default.

### 10.0.3 A.3 Backend Setup

```
# Navigate to backend directory
cd gym-management-system-fullstack/backend

# Configure database in application.properties:
# spring.datasource.url=jdbc:oracle:thin:@localhost:1521:orcl
```

```
# spring.datasource.username=YOUR_DB_USERNAME
# spring.datasource.password=YOUR_DB_PASSWORD


# Build and run
./mvnw spring-boot:run
```

The backend API server starts on `http://localhost:8080` by default.

### 10.0.4 A.4 Environment Variables Reference

| Variable | Description | Example |
|---|---|---|
| `VITE_API_BASE_URL` | Backend API URL | `http://localhost:8080/api` |
| `VITE_WS_URL` | WebSocket server URL | `ws://localhost:8080/ws` |
| `spring.datasource.url` | JDBC connection string | `jdbc:oracle:thin:@...` |
| `jwt.secret` | JWT signing secret (min 256-bit) | `[secure-random-string]` |
| `spring.mail.host` | SMTP host for email OTP | `smtp.gmail.com` |

## Appendix B: Database Schema Reference

### 10.0.5 B.1 Core Tables Summary

| Table Name | Primary Key | Description |
|---|---|---|
| `users` | `user_id` | All user accounts across roles |
| `roles` | `role_id` | Role definitions (MEMBER, TRAINER, OWNER, ADMIN) |
| `gyms` | `gym_id` | Gym tenant records |
| `memberships` | `membership_id` | Active and historical memberships |
| `membership_packages` | `package_id` | Configured membership packages |
| `pt_sessions` | `session_id` | Personal training session bookings |
| `trainer_details` | `trainer_id` | Trainer profile extensions |
| `conversations` | `conversation_id` | Chat conversation threads |
| `messages` | `message_id` | Individual chat messages |
| `notifications` | `notification_id` | System notifications |

### 10.0.6 B.2 Key Relationships

- Each `users` record is associated with one or more `roles` via a join table.

- Each `gym` has exactly one `owner` (foreign key to `users`).
- Each `membership` references one `membership_package` and one `user`.
- Each `pt_session` references one trainer `user`, one member `user`, and belongs to one `gym`.
- Each `conversation` contains zero or more `messages`, and involves two or more `users`.

## Appendix C: API Endpoints Reference

### 10.0.7 C.1 Authentication Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/auth/register` | Register a new user |
| POST | `/api/auth/login` | Authenticate and receive JWT |
| POST | `/api/auth/refresh` | Refresh expired JWT |
| POST | `/api/auth/logout` | Invalidate session |
| POST | `/api/auth/verify-otp` | Verify 2FA OTP |

### 10.0.8 C.2 Membership Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/memberships/packages` | List available packages |
| POST | `/api/memberships/request` | Submit membership purchase request |
| PUT | `/api/memberships/{id}/approve` | Owner approves membership |
| PUT | `/api/memberships/{id}/reject` | Owner rejects membership |
| GET | `/api/memberships/my` | Get current member's membership |

### 10.0.9 C.3 PT Session Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/sessions/trainers` | Browse available trainers |
| POST | `/api/sessions/book` | Book a PT session |
| PUT | `/api/sessions/{id}/complete` | Mark session as complete |
| PUT | `/api/sessions/{id}/cancel` | Cancel a booked session |
| POST | `/api/sessions/{id}/rate` | Submit session rating |

## Appendix D: Technology Dependencies

### 10.0.10 D.1 Frontend Dependencies (Selected)

| Package | Version | Licence |
|---|---|---|
| react | 18.x | MIT |
| react-dom | 18.x | MIT |
| react-router-dom | 6.x | MIT |
| axios | 1.x | MIT |
| socket.io-client | 4.x | MIT |
| framer-motion | 10.x | MIT |
| recharts | 2.x | MIT |
| lucide-react | latest | ISC |
| typescript | 5.x | Apache-2.0 |
| vite | 5.x | MIT |

### 10.0.11 D.2 Backend Dependencies (Selected)

| Package | Version | Licence |
|---|---|---|
| spring-boot-starter-web | 3.x | Apache-2.0 |
| spring-boot-starter-security | 3.x | Apache-2.0 |
| spring-boot-starter-data-jpa | 3.x | Apache-2.0 |
| spring-boot-starter-websocket | 3.x | Apache-2.0 |
| jjwt-api | 0.11.x | Apache-2.0 |
| lombok | 1.18.x | MIT |
| ojdbc8 | 21.x | Oracle |
| spring-boot-starter-mail | 3.x | Apache-2.0 |